ARTICLE | Article in Press

Hybrid rank fusion of a group-token transformer with gradient boosting improves detection of evasive and novel-family malware

Mohammed Tawfik · Belal Al-sellami · Mohamed Heshmat · Islam S. Fathi · Warda M. Shaban

Received: 5 September 2026 / Accepted: 5 October 2026
© The Author(s) 2026

Abstract

Static machine-learning malware detectors report near-perfect closed-set accuracy, but the malware that matters operationally is the malware that slips past detection and the malware that belongs to families never seen in training. We study both on three longitudinal benchmarks: EMBER2024 (3.2 M Windows, .NET, APK, ELF and PDF files, 52 training weeks and 12 test weeks, plus a *challenge set* of 6,315 files that evaded every antivirus engine at first sight), LAMDA (1.0 M Android applications, 2013–2025, with an IID/NEAR/FAR drift protocol) and BODMAS (134 K PE files, 2019–2020, monthly evaluation). We make five contributions. First, an *operational* evaluation protocol: true-positive rate at 1% and 0.1% false-positive rate, challenge-set detection at fixed FPR, novel-family, singleton and low-consensus strata, week-by-week and year-by-year drift, thresholds calibrated on time-aware validation data, and paired-bootstrap confidence intervals. Second, three new deep architectures for static feature vectors, trained jointly across formats: GT-Former, a Group-Token Transformer over the semantic feature groups, whose hyper-parameters are tuned with Optuna on a time-aware validation window and which is evaluated with five seeds on every benchmark; FR-MoE, a format-routed mixture of experts; and ProtoCon-Net, a prototype classifier with family-aware supervised-contrastive training. Third, three metadata-driven objectives that use antivirus consensus and family labels at training time only (a consensus-aware margin loss, a consensus-regression head and a family-adversarial gradient-reversal head), with full ablations. Fourth, explainability with SHAP and CLS attention. Fifth, benchmark-hygiene findings, including that the public EMBER2024 release lists every record twice. The results are sobering and useful: the EMBER2024 LightGBM benchmark remains the strongest single model at strict operating points (Win32: 90.0% TPR at 0.1% FPR versus $\mathbf{88.1 \pm 0.4}$% for the tuned GT-Former over five seeds), and the metadata-driven objectives do not pay for themselves: on LAMDA, with five seeds per variant, no objective activated on its own is distinguishable from the plain model, and only their combination degrades the strictest operating point (NEAR TPR at 0.1% FPR falls from $\mathbf{10.1 \pm 2.3}$% to $\mathbf{4.4 \pm 3.7}$%). The seed spread on this benchmark is large enough that single-seed ablations are unreliable, which we regard as a methodological finding in itself. The deep models are nevertheless complementary to gradient boosting: a rank average of the tuned GT-Former and LightGBM raises Win32 challenge-set detection from 63.2% to 68.8% at 1% FPR (95% CI $+\mathbf{4.7}$ to $+\mathbf{6.6}$

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-75034-w

Published online: 12 October 2026

scientific reports

points), novel-family detection from 89.1% to 91.8% (CI +1.5 to +3.9) and test TPR at 1% FPR from 96.8% to 97.3%, with significant challenge-set gains also on .NET (+1.6) and APK (+3.9); on LAMDA a rank average of an MLP and LightGBM raises far-future (2018–2025) detection from 40.5% to 54.4% at 1% FPR (CI +13.7 to +14.0) and singleton-family detection from 34.6% to 53.3% *when the threshold is calibrated on the benign files of the test region*; under a threshold fixed on validation data and applied unchanged, twelve years of drift move both models far below their nominal budget and the advantage falls to 0.1 points (Section Retrospective and deployment operating points), which we regard as the honest deployment figure; and on BODMAS the tuned GT-Former/LightGBM fusion matches LightGBM at 1% FPR (99.85%) and improves detection at 0.1% FPR (98.9% to 99.5%) and novel-family detection at 0.1% FPR (94.7% to 97.1%). SHAP shows why the benchmark misses evasive files: its strongest evidence is structural sloppiness (suspicious section flags, missing signatures), and evasive malware is exactly the malware that avoids it. Code, de-duplication masks, models and all score files are released.

Keywords Malware detection · EMBER2024 · LAMDA · BODMAS · Evasive malware · Concept drift · Transformer · Mixture of experts · Contrastive learning · Gradient reversal · Explainable AI · Benchmark evaluation

Introduction

Static analysis is the first line of defence against malware because it is cheap and safe, and machine-learned classifiers over static features are its workhorse. Three longitudinal benchmarks now make it possible to ask how such classifiers behave against the malware that actually matters. EMBER2024[1] contains 3.2 million files of six formats collected week by week from September 2023 to December 2024 and, uniquely, a *challenge set* of 6,315 malicious files that no VirusTotal engine detected when they were first uploaded. LAMDA[2] contains one million Android applications spanning twelve years, with family labels for 1,380 families and 150,604 singletons, and a protocol that trains on 2013–2014 and tests on NEAR (2016–2017) and FAR (2018–2025) years. BODMAS[3] contains 134,435 timestamped PE files (August 2019 to September 2020) with 581 curated families and serves as a third, older-feature-version benchmark for generalisation.

On all three benchmarks the closed-set numbers are excellent and the operational numbers are not. The EMBER2024 LightGBM baselines reach ROC-AUC above 0.987 on every format but only 0.24–0.81 PR-AUC on the challenge set; the F1 of LightGBM on LAMDA falls from 97.5% in distribution to 47.2% on the FAR years. The benchmark authors themselves call for work on evasive malware and novel families.

This paper asks four questions. The first is how evasive-malware and drift robustness should be measured. AUC and PR-AUC over a mixture of challenge malware and benign files, or F1 at a fixed 0.5 threshold, do not correspond to operating points that a deployed detector uses; we therefore measure detection at 1% and 0.1% FPR, with thresholds set on test benign files (comparable to prior work) or on time-aware validation data (deployment-realistic), and stratify malware by family novelty, singleton status and antivirus consensus. The second question is whether modern deep architectures for static feature vectors close the gap to gradient boosting, and whether they are complementary to it; we design three architectures with different inductive biases, tune the main one (GT-Former with Optuna on time-aware validation data, train each jointly across formats and report five seeds. The third question is whether the metadata that ships with the benchmarks (antivirus detection counts and family labels) can be turned into training signals for evasive and novel-family malware; we propose three objectives and ablate them on two benchmarks. The fourth question is what the detectors actually look at and why they miss evasive files, which we answer with SHAP on the tree benchmark and attention analysis on GT-Former, in the spirit of the explanation-driven analyses of Aljarrah et al.[4] and Tawfik et al.[5].

The contributions of the paper follow from these questions. We introduce an operational evaluation protocol with stratified, per-period and calibrated-threshold reporting and paired-bootstrap confidence intervals, and apply

Table 1 Refereed studies reporting results on EMBER2024, LAMDA and BODMAS. Chall. = challenge set; "raw release" = trained on the Hugging Face files without de-duplication (Section EMBER2024). The last column states whether the study reports fixed-FPR detection / novel-family detection / challenge-set detection at a fixed FPR.

Study (venue, year)	Data and protocol	Models	Headline numbers	Fixed-FPR / novel / chall.@FPR
EMBER2024				
Joyce et al.[1] (KDD 2025)	all formats, test + challenge; official 52/12-week split	LightGBM per format	ROC-AUC 0.9868 (APK) to 0.9989 (Win64); challenge PR-AUC 0.237 (APK) to 0.807 (.NET), 0.665 (Win32)	no / family task only / no
Horvath and Zsigmond[11] (ICAART 2026)	PE subset (Win32, Win64, .NET), raw release; official split	LightGBM, XGBoost, CatBoost, RF, ExtraTrees, MLP	tuned LightGBM test ROC-AUC 0.9986, challenge ROC-AUC 0.9696 / PR-AUC 0.688; notes a model with high challenge AUC but 0% TPR at 0.1% FPR	TPR@0.1% on test only / no / no
Mann et al.[12] (ICAISET 2026)	Win32, 586 selected features; three chronological cut-offs (Dec. 2023, Jan. 2024, Mar. 2024)	SGD-LR, SGD-SVM, LightGBM, XGBoost, MLP	LightGBM/XGBoost peak AUC 0.9922 under chronological splits versus 0.9996 with random splits; linear recall falls from 0.898 to 0.726	no / no / no
Kalný et al.[13] (SECRYPT 2026)	family subsets over temporal windows; drift detection from decision-tree rule sets	RIPPER/tree rules versus Transcendent	two-month windows with feature-level Pearson correlation are the most reliable drift signal	not a detector study
Park et al.[14] (PAKDD 2026)	EMBER2018 and EMBER2024; class-incremental family classification	CTGAN generative replay + SimHash compression	more than 10% accuracy over MalCL-style baselines	no (family task)
Raza et al.[15] (IEEE Access 2026)	2,381 of the features; random split	GA feature selection + AutoML	96.9% accuracy, ROC-AUC about 0.99 with about 400 features	no / no / no
Mossadaq and Mandar[16] (ICCSDFAI 2026)	test + challenge; official split, three-class triage	1D-CNN + anomaly detectors (kNN distance)	auto-clear 98.6% pure, auto-block 99.0% precise, about 72% of files decided automatically; kNN fusion raises challenge recall from 16.9% to 43.7%	no / no / recall only (threshold unspecified)
Imran et al.[17] (Future Internet 2026)	test + challenge; official split, conformal gating with an FPR budget	LightGBM + out-of-fold rescue detector	recovers 178 extra test malware for 82 extra false positives and 17 extra challenge files	yes (FPR budget) / no / absolute counts only
Nguyen Kim et al.[18] (CMC 2026)	large PE datasets incl. EMBER-style features; strict time-based split	multiple-instance + contrastive multi-view	ROC-AUC above 0.98, F1 about 0.96	no / no / no
Rahman et al.[19] (CAMLIS 2025)	EMBER (Windows) and AZ (Android); continual learning with replay	distribution-aware replay	large gains over generic replay baselines	continual-learning metrics
LAMDA				
Haque et al.[2] (ICLR 2026)	LAMDA Baseline (4,561 features); TRAIN+IID 2013–14, NEAR 2016–17, FAR 2018–25; five seeds	linear SVM, LightGBM, MLP, XGBoost	LightGBM F1 97.49 (IID), 59.48 ± 28.20 (NEAR), 47.24 ± 27.33 (FAR); FNR 1.7% to 64.1%; MLP 97.21 / 56.57 / 47.59	no / no / –
Qiu et al.[20] (The Computer Journal 2026)	decade-long Android corpus; active learning with pseudo-label correction	uncertainty-based label-type recognition	false-negative rate reduced from 76.0% to 47.6% on average versus MORPH	no / no / –
Ghiani et al.[21] (IEEE TIFS 2026)	ELSA, TESSERACT and AZ-Class (cites LAMDA); continual-learning updates	positive congruent training	halves security regression (3–6% of malware regress after updates)	no / no / –
Chow et al.[10] (IEEE SaTML 2026)	AndroZoo-derived corpora; dataset-curation methodology	–	five neglected dataset-quality factors change detector rankings	not a detector study
Minnei et al.[22] (CEUR 2025)	Android and Windows corpora; active + semi-supervised update strategies	LightGBM / MLP	up to 90% fewer manual annotations at comparable detection	no / no / –
BODMAS				
Yang et al.[3] (DLS 2021)	classifiers trained on EMBER 2017–18, SOREL-20M and UCSB; tested month by month on BODMAS Oct. 2019 to Sep. 2020	GBDT and DNN	EMBER-trained GBDT: F1 92.1% to 97.8% by month at about 0% FPR (validation 98.6%); UCSB-GBDT falls to 47.2% (Aug. 2020); false negatives dominated by unseen families	yes (approx. 0% FPR) / qualitative / –

Table 1 (continued)

Study (venue, year)	Data and protocol	Models	Headline numbers	Fixed-FPR / novel / chall.@FPR
Wang et al.[24] (TrustCom 2022)	family classification on the 581 families	static and dynamic features, ML classifiers	static features effective even with 10% packed samples; heavy family imbalance; results depend on the evaluation methodology	no / no / –
Yan et al.[26] (QRS 2022)	label-bias study on 52,793 samples and 67 AV engines	label aggregation strategies	recommended aggregation raises F1 from 84.83% to 92.62%	no / no / –
Lu et al.[25] (IEEE Access 2022)	BODMAS-11 and BODMAS-49 family subsets	self-attention transformers on bytes/images	higher accuracy than CNN baselines at lower latency	no / no / –
Yumlembam et al.[27] (Knowledge-Based Systems 2025)	EMBER, BODMAS and UCSB under dataset shift	LightGBM + NN ensembles + conformal evaluation	incorrect-acceptance rate on UCSB reduced from 22.8% to 16%	no / no / –
Huynh et al.[23] (IEEE Access 2026)	1.68 M PE files from EMBER 2017, EMBER 2018 and BODMAS; forward/backward temporal splits	LightGBM, RF, MLP	8.47-point F1 loss from 2017 to 2018; false-negative rate up to 23.92% driven by unseen families; incremental retraining with 1% new labels recovers 0.56 points AUT	no / qualitative / –
Truong[28] (IEEE Access 2026)	cross-dataset transfer among EMBER, BIG15, Malicia, BODMAS	RF, MLP, LightGBM, OC-SVM	RF mean F1 98.72%, AUC 0.998 in distribution; feature overlap 7.9–27.6% across datasets, 83.8% between EMBER and BODMAS	no / no / –

it to EMBER2024, LAMDA and BODMAS (Sections Evaluation protocol and Results). We propose three new deep architectures, namely GT-Former (2.13 M parameters by default and 7.9 M after Optuna tuning), FR-MoE (5.28 M) and ProtoCon-Net (3.65 M), with a plain MLP and the official LightGBM as references, together with an Optuna study of GT-Former on each benchmark (Sections Architectures and Training protocol). We formulate three metadata-driven objectives (a consensus-aware margin, a consensus head and a family gradient-reversal head) and present complete ablations, including add-one-in runs over five seeds showing that no objective is individually distinguishable from the plain model and that only their combination degrades the strictest operating point (Sections 4.1 and Operational metrics under drift). We show that hybrid rank fusion of a deep model with LightGBM gives statistically significant gains where they matter: +5.6 points of challenge-set detection and +2.7 points of novel-family detection on EMBER2024 Win32, +13.8 points of far-future detection and +18.7 points of singleton detection on LAMDA at region-calibrated thresholds (but not at a fixed deployment threshold, Section Retrospective and deployment operating points), and +0.5 points at 0.1% FPR and +2.5 points of novel-family detection at 0.1% FPR on BODMAS (Sections 4.1, 4.2 and 4.7). Explainability reveals a sign flip of the benchmark's most important features between ordinary and evasive malware and a shift of GT-Former's attention to export tables and strings for evasive files (Section Explainability: what separates evasive from ordinary malware). Finally, we report benchmark-hygiene findings: the exact duplication of every record in the public EMBER2024 release, a characterisation of the challenge set as low-consensus, family-less malware, and reproductions of the LAMDA and BODMAS protocols with released splits (Section Datasets).

Related work

EMBER[6] established feature-vector benchmarks for Windows PE malware and a LightGBM[7] baseline that subsequent work has found hard to beat. EMBER2024[1] is its successor: 3.2 M files of six formats collected week by week from September 2023 to December 2024, feature version 3 (2,568 dimensions, pefile-based, with DOS-header, rich-header, data-directory, Authenticode and parser-warning features), seven labelled tasks, and a challenge set of 6,315 files that were undetected by every VirusTotal engine at first submission. BODMAS[3] provides

134,435 timestamped PE files (August 2019 to September 2020) with EMBER-version-2 features and 581 analyst-curated families. LAMDA[2] is the largest longitudinal Android benchmark: 1,008,381 AndroZoo APKs from 2013 to 2025 (excluding 2015), Drebin-style features reduced to 4,561 binary dimensions, VirusTotal detection counts, AVClass2 families (1,380 families, 150,604 singletons). Family counts on LAMDA depend entirely on whether singleton labels are counted as families, which is the source of the apparent inconsistency between the benchmark's headline figure and per-year counts. Recounting the released Baseline metadata, the 369,906 malware carry 151,985 distinct family strings, of which 151,075 occur exactly once and only 910 occur more than once; the benchmark's 1,380 refers to named families with repeated members and the singletons are reported separately. Per-year counts in this paper follow the same convention as the benchmark's own drift tables and count distinct labels, so 2017 has 21,359 malware carrying 9,271 distinct labels of which 9,146 are singletons, i.e. 125 families with more than one member. We state the convention with every count to avoid the ambiguity and a training/IID/NEAR/FAR protocol. Temporal bias in malware evaluation was formalised by Transcend[8] and TESSERACT[9]; Chow et al.[10] show that discrepancies persist even under time-aware evaluation and propose a trustworthy curation methodology for AndroZoo-derived datasets.

Table 1 lists every refereed study we could locate that reports results on one of the three benchmarks (from the Semantic Scholar citation graphs of the three dataset papers, checked in September 2026), with its data subset, protocol, models and headline numbers; preprints that have not appeared in a refereed venue are not cited. On EMBER2024, the studies range from tree-model benchmarks on the PE subset[11] and chronological-split analyses of Win32[12] to drift detection from rule sets[13], class-incremental family classification[14], evolutionary feature selection with AutoML[15], SOC triage with anomaly detectors[16], conformal gating under a false-positive budget[17], multiple-instance contrastive learning[18] and distribution-aware replay for continual learning[19]. Two gaps are visible. First, no refereed study reports challenge-set detection at a fixed false-positive rate, stratifies by family novelty or antivirus consensus, or evaluates the twelve test weeks separately; the closest are C-GUARD, which reports absolute counts under an FPR budget, and the SOC triage study, which reports challenge recall without a stated operating point. Second, every study that trains on the public files trains on the duplicated release (Section EMBER2024). The PE-only tuned LightGBM of Horvath and Zsigmond[11] (challenge ROC-AUC 0.9696) and the per-format models of Joyce et al.[1] (Win32 challenge ROC-AUC 0.9689, PR-AUC 0.665) are therefore the reference points for our reproduction in Section Reproduction of the published baseline and for the fusion results in Section 4.1 (Win32 challenge ROC-AUC 0.9752, PR-AUC 0.670).

On LAMDA, to our knowledge no refereed study other than the benchmark paper itself reports detector results under its protocol; the closest refereed work addresses pseudo-label noise under drift[20], regression-aware continual learning[21], dataset curation[10] and label-efficient updates[22] on other Android corpora. Section LAMDA therefore provides the first external replication of the LAMDA protocol (LightGBM F1 97.47 / 73.1 / 58.7 for IID / NEAR / FAR, inside the benchmark's reported variance) and the first fixed-FPR, novel-family and singleton stratification on it. On BODMAS, the dataset paper[3] and the multi-year study of Huynh et al.[23] evaluate month by month with classifiers trained on EMBER or SOREL-20M; family classification[24,25], label bias[26], uncertainty under dataset shift[27] and cross-dataset feature discriminability[28] complete the picture. Our BODMAS protocol (train up to February 2020, validate on March, test April to September) follows the monthly evaluation of those studies; the numbers are not directly comparable because they train on other corpora, but the ordering of difficulty (August and September 2020 hardest, unseen families the main source of misses) is the same as in Section BODMAS.

Antivirus-label aggregation (AVClass2, ClarAVy) yields family names and confidence, and the detection count is normally used only as a labelling threshold (four or more engines in LAMDA, five or more in EMBER2024); Yan et al.[26] quantify how much label aggregation alone moves F1 on BODMAS. Semantic grouping of static features has recently been shown to be a productive inductive bias: Aljarrah et al.[4] organise EMBER-style features into structural and content groups fused by multi-head cross-attention and report robust zero-day generalisation under family-based splits, and Tawfik et al.[5] combine prototypical networks with drift detection and explainability for few-shot Android family classification; both, however, evaluate at a fixed threshold rather than at an operational false-positive budget. The same pairing of few-shot learning with explainability has been applied to

other evolving-threat domains, with cross-attention and federated training over medical-IoT traffic[29] and with a transformer encoder and prototypical/MAML meta-learning over phishing URLs[30]. Those settings share the problem this paper studies, namely detecting threat classes that were absent or nearly absent at training time, but they operate on network-flow and URL features and report accuracy at a fixed decision threshold; here the evidence is a static binary feature vector and the question is what survives at a 1% and 0.1% false-positive budget, where, as Section Results shows, the same architectural ideas behave very differently. Supervised contrastive learning[31] and domain-adversarial training with gradient reversal[32] have been applied to drift and cross-dataset transfer, but no refereed work we found uses the antivirus consensus as a margin or weighting signal, or removes family identity adversarially, for evasive or novel-family malware on these benchmarks; Sections 4.1 and Operational metrics under drift show that, as designed, both hurt.

Static feature engineering for PE malware

A parallel line of work optimises the feature representation rather than the classifier. Singh et al.[33] select portable-executable header fields and show that a compact header subset retains most of the discriminative signal; the same group derives deep features with a feed-forward network[34], minimises feature overhead by MRMR ranking[35], raises SVM detection accuracy by preprocessing[36], and characterises the CLaMP corpus[37]. These studies evaluate a few thousand files at a fixed decision threshold. The question here is different: what survives at a 0.1% false-positive budget on 3.2 million files collected week by week. Our group tokens therefore keep the whole EMBER version-3 vector and let attention learn which semantic groups matter, instead of discarding features in advance; Section Explainability: what separates evasive from ordinary malware shows that the groups the model relies on change between ordinary and evasive malware, which a fixed selected subset cannot express.

Forensic and cross-domain malware analysis

Beg et al.[38] localise malware across domains with an augmented convolutional model and forensic neural networks; a later iterative design combines autoencoders with hybrid machine-learning models[39], an extreme-learning model detects malware evidence from static and dynamic information for forensic analysis[40], and an evidence-driven few-shot meta-learner performs zero-day detection with forensic attribution[41]. Our setting differs in three respects that matter for the comparison. We use static feature vectors only, with no dynamic trace, sandbox execution or disassembly, so the evidence available to the model is strictly poorer. We evaluate on longitudinal public benchmarks at fixed false-positive budgets with time-aware splits, rather than on curated cross-domain corpora at a fixed threshold. And we do not attempt attribution: family labels enter only as a training-time signal, which Section Activating each objective on its own shows is indistinguishable from the plain objective when used alone and harmful only in combination with the consensus objectives, and as an evaluation stratum for novel-family detection.

Methods from adjacent application domains

Three mechanisms used in this paper have close analogues outside malware detection, and the comparison is instructive because the constraints differ. Attention combined with metaheuristic hyper-parameter search has been applied to rare-attack detection in IIoT networks[42], a security setting that shares our extreme class imbalance but not our temporal drift. Calibrated, cost-sensitive thresholding with validation-selected operating points is central to verifier-grounded transaction-fraud triage[43]; that argument is the same one we make in Section Retrospective and deployment operating points, namely that an operating point chosen on the evaluation data is not a deployment result. Attention-guided fusion of heterogeneous backbones[44] combines representations inside one network, whereas our rank fusion combines two independently trained models at the score level, which is what allows a gradient-boosted tree to be one of the partners; and self-supervised pretraining that reduces label

dependence[45] is a direction we consider promising for novel families, where labels are by definition unavailable. None of these works addresses malware, evasion or concept drift, and none is evaluated at a fixed false-positive budget, so they are context for the mechanisms rather than baselines for the task.

Relative to Table 1, this paper is therefore the first to report challenge-set and novel-family detection at fixed operating points with confidence intervals on EMBER2024, to evaluate one deep architecture family across six file formats jointly and on three benchmarks, to tune the deep model with a documented Optuna study and five seeds, to reproduce the LAMDA protocol externally, and to document and correct the duplication of every record in the public EMBER2024 release.

Methodology

This section describes the three benchmarks and their protocols (Section Datasets), the preprocessing and the supervision available at training time, the objectives under study, the GT-Former architecture and its two alternatives (Figure 1), the training protocol, and the operational evaluation protocol.

Datasets

We use three public longitudinal benchmarks that together cover Windows, .NET, Linux, PDF and Android malware, two feature layouts (EMBER versions 2 and 3, and Drebin-style binary indicators) and three notions of difficulty: an evasive challenge set, a twelve-year drift protocol, and a monthly PE protocol with unseen families.

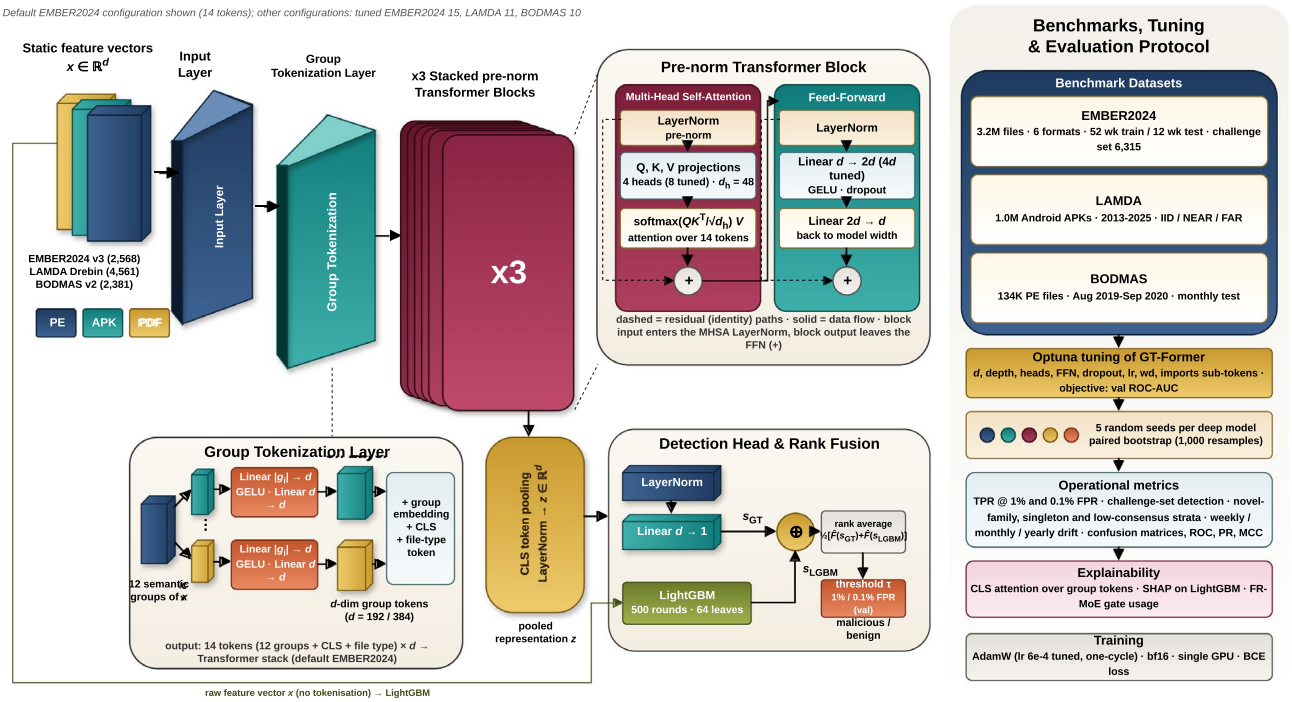


Fig. 1 GT-Former architecture. Static feature vectors from the three benchmarks pass through the input layer and the Group Tokenization Layer (detail, bottom left), three stacked pre-norm Transformer blocks (detail, top centre), CLS pooling and the detection head (LayerNorm, linear $d \to 1$); the detection logit is rank-fused with LightGBM, which receives the raw feature vector directly (bypass line), and thresholded at a fixed false-positive rate. Right: benchmarks, Optuna tuning, seeds and the evaluation protocol. Token counts in the figure are those of the default EMBER2024 configuration (12 group tokens, a file-type token and CLS: 14); the tuned EMBER2024 model splits the imports group into two tokens (15), LAMDA uses the ten Drebin categories and CLS with no file-type token (11), and BODMAS uses the nine version-2 groups and CLS (10).

EMBER2024

We downloaded the complete release (24.3 GB of JSONL in zip archives) from Hugging Face (`joyce8/EMBER2024`, Apache-2.0 licence) and vectorised every record with the authors' `thrember` feature extractor (feature version 3, 2,568 dimensions). Each JSONL member contains *every record exactly twice* as identical lines. Raw row counts are therefore twice the paper's: 720,000 rows for the Win32 test set versus 360,000 files. De-duplicating by SHA-256 (first occurrence) recovers the published sizes exactly, namely 2,626,000 training and 606,000 test files (a handful of files that appear in two weeks are also collapsed, for example 359,994 unique Win32 test files); the 6,315 challenge files are not duplicated. The duplicates never cross the train/test boundary, so prior results are not invalidated, but any count, class-balance statistic, training time or per-epoch cost reported from the raw files is inflated by a factor of two. Table 2 gives the de-duplicated statistics.

The novel-family stratum is defined *within format*: a test file is novel if its ClarAVy family does not occur among the training malware of its own format. This is the natural definition for LightGBM, which is trained per format, but the deep models are trained jointly on all six formats, so a family that is novel to the Win64 training malware may have been seen during training under Win32 or .NET. For the jointly trained models and for the fusion, within-format novelty is therefore a weaker criterion than global novelty, and the novel-family numbers we report for them should be read as within-format results; the effect can only favour the deep models, so it does not weaken the comparison in the direction of our claims, but it does mean that novel-family detection on EMBER2024 is not a measurement of zero-day family detection in the global sense.

The EMBER2024 challenge set

The 6,315 challenge files are 51% Win32, 13% .NET, 13% Win64, 13% PDF, 6% ELF and 4% APK. Their *final* detection ratio (after rescans) has median 0.14 (inter-quartile range 0.09–0.23), against roughly 0.5–0.8 for ordinary training malware; 47% carry no ClarAVy family at all; and 85% were first seen during the *training* period (weeks 0–51), so they are contemporaneous with the training data rather than temporally novel. "Evasive" in EMBER2024 therefore means *low-consensus and family-less*, which is exactly the stratum that a standard cross-entropy objective, dominated by well-detected samples of large families, sees least of.

LAMDA

We use the Baseline variant of LAMDA from Hugging Face (`IQSeC-Lab/LAMDA`, MIT licence): 1,008,381 unique SHA-256 hashes (no duplicates), 369,906 malware (36.7%), 4,561 binary Drebin-style features in ten categories (URL domains 1,403; activities 1,395; intent filters 402; services 402; requested permissions 341; broadcast receivers 313; restricted API calls 175; suspicious API calls 47; hardware components 46; used permissions 37), a VirusTotal detection count (median 8 for malware, IQR 6–14; always 0 for benign), an AVClass2 family (150,604 malware, 40.7%, are singletons) and a year-month stamp. The malware share collapses in the last years (48% in 2022, 14.5% in 2023, 1.6% in 2024, 0.1% in 2025), which is part of the drift the benchmark measures.

Table 2 EMBER2024 after de-duplication. Families = distinct ClarAVy families among malware; no-family = fraction of malware without a family label; det. ratio = median final antivirus detection ratio of malware; novel = fraction of test malware whose family never occurs in training malware of the same format (within-format novelty; see text).

Format	Train files	Test files	Train families	No-family (train)	Median det. ratio (train / test)	Novel-family test malware
Win32	1,560,000	359,994	4,437	12.1%	0.78 / 0.77	0.5%
Win64	520,000	119,993	1,997	20.5%	0.51 / 0.36	2.0%
.NET	260,000	59,953	1,714	21.5%	0.57 / 0.56	0.7%
APK	208,000	48,000	822	20.1%	0.21 / 0.23	0.4%
ELF	26,000	5,989	264	6.5%	0.50 / 0.49	0.9%
PDF	52,000	12,000	84	78.0%	0.38 / 0.26	0.7%

Following Haque et al.[2], TRAIN is all samples of 2013 and 2014 except the last collected month of each year (2013-12 and 2014-08 form the IID test set of 15,286 files); we additionally hold out a random 15% of TRAIN as a validation set for early stopping and model selection (25,849 files), leaving 146,479 training files. NEAR is 2016–2017 (208,337 files) and FAR is 2018–2025 (612,430 files).

BODMAS

BODMAS[3] provides EMBER-version-2 feature vectors (2,381 dimensions, LIEF-based) for 57,293 malware and 77,142 benign PE files with first-seen timestamps, 581 families and six categories. Malware spans August 2019 to September 2020; benign files reach back to 2007. We use a temporal protocol in which TRAIN is every file first seen up to February 2020 (65,559 files, 24,602 malware), VAL is March 2020 (8,567 files) and TEST is April to September 2020 (60,309 files, 27,701 malware, 1,215 of them from families unseen in training). BODMAS carries no antivirus consensus, so only the plain objectives apply, and the group tokens follow the version-2 layout (histogram, byte-entropy, strings, general, header, sections, imports, exports, data directories).

Preprocessing

Tree models use the raw EMBER version-3 vectors and the raw binary LAMDA vectors. For the deep models each EMBER vector x is transformed component-wise as

$$\tilde{x}_j = \mathrm{clip}\left(\frac{\mathrm{sign}\,(x_j)\,\log(1+|x_j|)-\mu_j}{\sigma_j},\,-12,\,12\right), \tag{1}$$

where μ_j and σ_j are estimated on a 60,000-file sample per format, and the result is stored in fp16; LAMDA vectors are used as they are. Seven integer fields (indices 2–6, 701 and 702) are declared categorical for LightGBM, as in the benchmark code.

Supervision available at training time

Each training file carries a label $y \in \{0,1\}$, a file type t (EMBER only), an antivirus consensus r (EMBER: the final detection ratio; LAMDA: $\min(\mathrm{vt_count}/30,1)$) and a family f (EMBER: ClarAVy families with at least 50 training files across formats, $|V| = 1{,}329$; LAMDA: AVClass2 families with at least 20 training files, singletons excluded). Neither r nor f is used at inference. We define an evasiveness score

$$e = \begin{cases} \mathrm{clip}\,(1-r/r_{95},\,0,\,1) & \text{(EMBER)},\\ 1-r & \text{(LAMDA)}, \end{cases} \qquad e = 0 \text{ for benign files}, \tag{2}$$

where r_{95} is the 95th percentile of r over training malware.

Objectives

All deep models minimise

$$\mathcal{L} = \mathcal{L}_{\mathrm{det}} + \lambda_{\mathrm{aux}}\mathcal{L}_{\mathrm{aux}} + \lambda_{\mathrm{grl}}\mathcal{L}_{\mathrm{fam}}\ (+\lambda_{\mathrm{con}}\mathcal{L}_{\mathrm{con}} \text{ for ProtoCon-Net}). \tag{3}$$

In the *plain* configuration $\mathcal{L}_{\mathrm{det}}$ is binary cross-entropy (with a positive-class weight equal to the benign/malware ratio on LAMDA) and the other terms are switched off. The *full* configuration switches on three metadata-driven terms. The first is the consensus-aware margin (CAM): for a file with logit z, the detection loss becomes

$$\mathcal{L}_{\text{det}} = \frac{1}{N}\sum_{i=1}^{N}(1+\lambda_w e_i)\,\text{BCE}(z_i - m\,e_i\,y_i,\ y_i), \qquad m = 2,\ \lambda_w = 1, \tag{4}$$

so that low-consensus malware must be separated by a larger margin and receives more gradient. The second is a consensus head: $\sigma(h_{\text{aux}}(z))$ regresses r on malware with a mean-squared-error loss and $\lambda_{\text{aux}} = 0.5$. The third is a family-adversarial head: a classifier over V is attached through a gradient-reversal layer[32] with the standard sigmoid ramp of the reversal coefficient and $\lambda_{\text{grl}} = 0.1$, so that the encoder is pushed to discard family-specific cues. For LightGBM we test the tree analogues on EMBER2024: per-sample weights $1 + 3e$ (consensus weights), $1/\sqrt{\text{family count}}$ normalised to mean one (family-balanced weights), both together, and an inverse-consensus control that up-weights *well-detected* malware.

Architectures

Figure 1 shows the GT-Former architecture together with the benchmark, tuning and evaluation protocol; the two alternative encoders and the reference models are described afterwards. Read end to end, the pipeline is: a static feature vector is standardised (Section Preprocessing); the group tokenisation layer turns its semantic groups into a short sequence of tokens, which is what allows attention to weigh, for instance, the export table against the byte histogram for a given file; three pre-norm Transformer blocks mix the tokens and a CLS token summarises them; a linear head produces a logit, which during training receives the plain cross-entropy loss and, in the ablated variants only, the consensus and family objectives of Section Objectives; at inference the logit is mapped to a validation percentile and averaged with the percentile of the LightGBM score of the same file (hybrid rank fusion), and the fused score is thresholded at a fixed false-positive budget. FR-MoE and ProtoCon-Net are alternative encoders that replace the tokenisation-plus-attention stage and are evaluated in the same way. Each stage is isolated by one experiment: the tokenisation by the MLP control at matched input and budget, the objectives by the leave-one-out and add-one-in ablations (Sections 4.1 and Activating each objective on its own), the fusion by the weight sweep and the paired bootstrap against LightGBM alone (Sections 4.1 and Sensitivity of the fusion to its weighting), and the percentile map by the retrospective-versus-deployment comparison (Section Retrospective and deployment operating points).

GT-Former (Group-Token Transformer) Following the observation that semantically grouped features fused by attention generalise better to unseen families than a flat encoder[4], the feature vector is split into its semantic groups $g_1, \ldots, g_G$ (EMBER: general 7, byte histogram 256, byte-entropy 256, strings 177, header 74, sections 224, imports 1,282, exports 129, data directories 34, rich header 33, Authenticode 8, pefile warnings 88; LAMDA: the ten Drebin categories). Each group is projected by a two-layer MLP to a d-dimensional token and summed with a learned group embedding E_i,

$$h_i = W_2\,\text{GELU}(W_1 \tilde{x}_{g_i} + b_1) + b_2 + E_i, \qquad i = 1, \ldots, G, \tag{5}$$

with $d = 192$ by default. On EMBER a file-type embedding token h_t is added and a learnable CLS token is prepended, giving $H_0 = [\,h_{\text{CLS}}; h_t; h_1; \ldots; h_G\,]$ with 14 tokens on EMBER and 11 on LAMDA. A three-layer pre-norm Transformer encoder[46] with four heads, feed-forward width 384 and dropout 0.1 processes the sequence,

$$H'_\ell = H_{\ell-1} + \text{MHSA}(\text{LN}(H_{\ell-1})), \qquad H_\ell = H'_\ell + \text{FFN}(\text{LN}(H'_\ell)), \tag{6}$$

and the CLS output after LayerNorm is the representation $z = \text{LN}(H_L[\text{CLS}]) \in \mathbb{R}^d$, from which a linear head produces the logit $s_{\text{GT}} = w^\top z + b$. The default model has 2.13 M parameters on EMBER.

FR-MoE (Format-Routed Mixture of Experts, EMBER only) A shared trunk ($2{,}568 + 32 \to 1{,}024 \to 512$, GELU, LayerNorm) feeds six experts ($512 \to 384 \to 192$) and a shared expert; a gate conditioned on the trunk output *and* the file-type embedding produces soft mixing weights. The model has 5.28 M parameters.

ProtoCon-Net Prototype-based classification has proved effective for scarce and drifting malware families[5]. An MLP encoder (input → 1,024 → 512 → 192, LayerNorm) is followed by $K = 4$ learnable prototypes per class, $\{p_{ck}\}$, and the detection logit is

$$s = \operatorname{logsumexp}_k \frac{\cos(z, p_{1k})}{\tau} - \operatorname{logsumexp}_k \frac{\cos(z, p_{0k})}{\tau}, \qquad \tau = 0.1. \tag{7}$$

A projection head (192 → 128) receives a supervised-contrastive loss[31] whose positives are same-family malware pairs or benign–benign pairs ($\lambda_{\text{con}} = 0.3$). The model has 3.65 M parameters on EMBER.

Hyper-parameter tuning of GT-Former Because a fixed default configuration is an unfair representative of an architecture, the width (128–384), depth (2–6), number of heads (4 or 8), dropout (0–0.3), learning rate (2×10^{-4} to 3×10^{-3}), weight decay (10^{-6} to 10^{-2}), feed-forward multiplier (2 or 4) and the number of sub-tokens for the 1,282-dimensional imports block (1, 2 or 4) of GT-Former were tuned with Optuna[47] using the TPE sampler: 20 trials on a 300 K-file, 3-epoch EMBER2024 proxy, 25 trials on LAMDA and 25 on BODMAS, with validation ROC-AUC on the time-aware validation split as the objective. ROC-AUC rather than TPR at 0.1% FPR was used as the objective because the latter is determined on a validation window by a handful of the highest-scoring benign files and varies by several points between seeds (Section 4.1), so a 20-trial search on it would mostly be fitting noise; the price is that the objective is not the metric we ultimately report, and Section Sensitivity of the fusion to its weighting and the seed statistics are the evidence that the tuned model's strict-FPR behaviour is not an artefact of that choice. The search was applied to GT-Former only; LightGBM keeps the published benchmark configuration and the MLP and FR-MoE keep their defaults, so the comparison between the tuned GT-Former and the other deep encoders is not budget-matched and we do not draw architectural conclusions from it (Section Discussion). The MLP does provide a parameter control for the default GT-Former: it has 3.58 M parameters against 2.13 M, uses the same input and the same training recipe, and differs only in replacing the semantic tokenisation and attention by a flat encoder, so differences between the two at equal budget are attributable to the tokenisation rather than to capacity. The selected EMBER2024 configuration is $d = 384$, 3 layers, 8 heads, feed-forward multiplier 4, imports split into two tokens, dropout 0.12 and learning rate 6×10^{-4} (7.9 M parameters); LAMDA selects $d = 384$, 3 layers and 8 heads; BODMAS selects $d = 192$, 3 layers and 8 heads. All tuning and training ran on a single GPU.

Reference models We use the EMBER2024 LightGBM configuration (500 rounds, 64 leaves, learning rate 0.1, `is_unbalance`), with one model per format on EMBER and one model on LAMDA, using the same validation data for early stopping; and a plain MLP with the ProtoCon-Net encoder and a linear head (3.58 M parameters on EMBER). Recent tabular deep learners such as TabM[48] and ModernNCA[49], both ICLR 2025, are stronger flat baselines than a plain MLP on general tabular benchmarks; we did not train them on these corpora, and the flat-encoder reference in this paper is therefore the MLP, whose role is to isolate the effect of the tokenisation at matched input, data and budget rather than to represent the best available tabular model. C-GUARD[17] has no public implementation, so it is compared on its published counts (Table 1) rather than re-run on our splits.

Hybrid rank fusion Each model's score is first mapped to a percentile through a fixed monotone map learned once on the validation scores of that model and then applied unchanged to every later file. Writing $\hat{F}_m$ for the empirical distribution function of model m on the validation files (ties resolved by the midpoint convention, so a score equal to a validation score receives the average of the ranks it could take), the fused score is

$$s_{\text{fus}} = \tfrac{1}{2}\left[\hat{F}_{\text{GT}}(s_{\text{GT}}) + \hat{F}_{\text{LGBM}}(s_{\text{LGBM}})\right], \tag{8}$$

So that the two models contribute equally regardless of their score scales. Because $\hat{F}_m$ is fixed before any test or challenge file is seen, the fused score of a file is a function of that file alone: it does not change when other files are added to or removed from the evaluation cohort, and the fusion can be applied one file at a time in chronological order. The submitted version instead ranked each model's scores over the pooled validation, test and challenge files, so a file's fused score depended on the cohort it was evaluated with; on Win32 the two definitions give

68.87% and 68.96% challenge detection at 1% FPR respectively, a difference of 0.09 points, and every fusion number in this version uses the fixed map. The fused score is then thresholded exactly like a single model.

Training protocol

On EMBER the deep models are trained jointly on all formats. The 2,626,000-file training partition is split time-aware into a fitting set and the last four collected weeks (weeks 48–51), which are held out for model selection and are never fitted on; the standardisation statistics are label-free per-feature means and standard deviations estimated from a 60,000-file random sample of each format's training partition, which includes those four weeks, and never from test or challenge files. Training uses format-balanced sampling (probability proportional to $1/\sqrt{n_{\text{format}}}$), AdamW (learning rate 10^{-3} for GT-Former and 5×10^{-4} otherwise, one-cycle schedule, weight decay 10^{-4}), batch size 2,048, 8 epochs, bf16 autocast and gradient clipping at 5; model selection uses ROC-AUC on the time-aware validation window (training weeks 48–51); five seeds (0–4) are run for GT-Former (default and tuned), the MLP and FR-MoE. On LAMDA we use the same optimiser with batch size 1,024, 12 epochs and model selection on the 15% validation split; five seeds for GT-Former (default and tuned) and the MLP, three for LightGBM. On BODMAS we use batch size 512, 15 epochs and five seeds for the tuned GT-Former. All experiments ran on one RTX A4000 Laptop GPU (8 GB); one EMBER deep run takes 8–25 minutes, one LAMDA run 1–3 minutes and one BODMAS run under a minute; the LightGBM reference on BODMAS uses the LightGBM GPU device.

Evaluation protocol

For a false-positive budget α, the operating threshold is the $(1-\alpha)$ quantile of the scores of benign files,

$$\tau_\alpha = Q_{1-\alpha}(\{s_i : y_i = 0\}), \qquad \text{TPR@}\alpha = \frac{1}{|\{i : y_i = 1\}|}\sum_{i:y_i=1} \mathbb{1}[s_i \geq \tau_\alpha], \tag{9}$$

with the benign set taken either from the test files (comparable to prior work) or from the validation files (deployment-realistic, with the realised test FPR reported). The closed-set metrics are ROC-AUC, PR-AUC and TPR at 1% and 0.1% FPR, reported per format on EMBER and per region and per year on LAMDA; on LAMDA we also report F1, FNR and FPR at the 0.5 probability threshold for comparability with Haque et al.[2], for single models only, since rank-averaged scores are not probabilities. Because all deep scores are raw logits, that decision rule is applied as $\sigma(s) > 0.5$, equivalently $s > 0$, while the LightGBM probabilities are thresholded at 0.5 directly; the two are the same rule. Challenge-set detection on EMBER is the fraction of challenge files of a format scored above the threshold giving 1% (or 0.1%) FPR on the test benign files of the same format, and also above the threshold calibrated on *validation* benign files. Malware is stratified into known-family, novel-family (family never seen in training malware) and no-family or singleton strata, and into a low-consensus stratum (bottom-quartile consensus on EMBER; at most six detections on LAMDA). Drift is measured as TPR at 1% FPR per test week on EMBER and per year on LAMDA, with the threshold fixed on the IID benign files. In absolute terms the two budgets mean the following on the largest format: the Win32 test set holds 179,994 benign files, so a 1% budget permits about 1,800 false positives (the validation-calibrated LightGBM threshold produces 1,669, Fig. 5) and a 0.1% budget about 180, and each point of TPR at that budget is 1,800 malware files detected or missed. A gain of five points of challenge-set detection at 1% FPR, as the fusion obtains on Win32, is therefore roughly 160 additional evasive files caught among the roughly 3,200 Win32 challenge files for no additional false positives, which is the scale at which the operational results below should be read. Uncertainty is quantified with a paired bootstrap (1,000 resamples) of the difference between each model and LightGBM on the same files, and as mean ± standard deviation over five seeds. Benign files, malware and challenge files are resampled separately and the operating threshold is *re-estimated inside every resample* from that resample's benign files, so the reported intervals include the uncertainty of the threshold itself and not only of the detection rate; intervals computed with a threshold fixed

once on the full sample are correspondingly narrower and we do not report them. Across the four pre-specified primary comparisons on Win32 (test TPR and challenge detection, each at 1% and 0.1% FPR) we apply the Holm step-down correction, and all four remain significant after it (Section 4.1).

All deep scores are raw logits. Probabilities saturate in fp32 and create ties at the top of the ranking, which silently destroys TPR at 0.1% FPR; in an early run we observed 21 Win32 benign files with logit above 17 that were indistinguishable after the sigmoid.

Results

We report the operational picture on each benchmark in turn: the reproduction of the published baseline, detection at fixed false-positive budgets on the evasive and novel-family strata, the complementarity of the deep models with gradient boosting, the ablation of the metadata-driven objectives, drift over time, seed variability, and the conventional classical metrics. Every number is computed from the released score files; confidence intervals are paired bootstraps against LightGBM on the same files.

EMBER2024

Reproduction of the published baseline

Our de-duplicated LightGBM models reproduce the published ROC-AUC closely: Win32 0.9982 (paper 0.9981), Win64 0.9988 (0.9989), .NET 0.9980 (0.9980), APK 0.9842 (0.9868), ELF 0.9845 (0.9933) and PDF 0.9906 (0.9912). The small deficits on ELF and PDF are consistent with our time-aware validation split (the benchmark used a random 10% split) and with halving the number of duplicated training rows.

Detection at fixed false-positive budgets

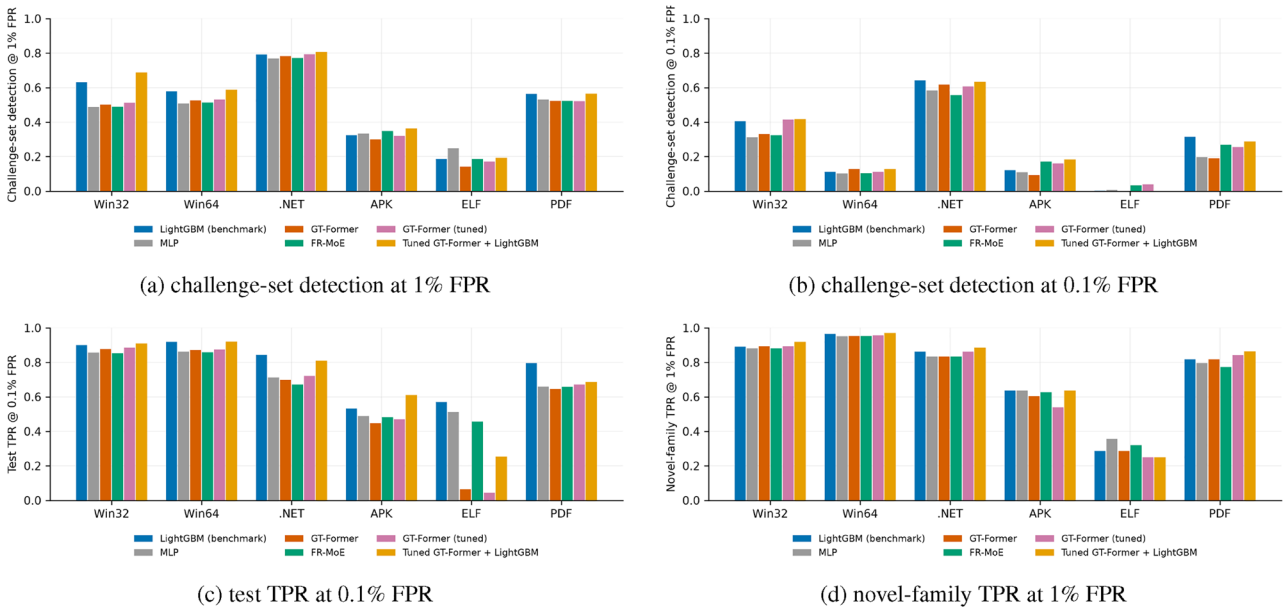
Table 3 gives the main comparison on the two largest formats, and Fig. 2 shows challenge-set detection at 1% and 0.1% FPR, test TPR at 0.1% FPR and novel-family TPR for every format. Three facts stand out. First, even the strongest single model detects only 63% of evasive Win32 malware and 58% of evasive Win64 malware at a 1% false-positive budget, and 40% and 11% respectively at 0.1%: the challenge set is far from solved. Second, novel-family malware is detected 8 points less often than known-family malware on Win32 (89.1% versus 97.5% for LightGBM), and on the small formats the gap is much larger (ELF: 28.6% versus 90.7%). Third, on its own every deep model trails LightGBM at strict operating points, by 1.5–5 points of TPR at 0.1% FPR on the PE formats and by 7–13 points on APK, ELF and PDF; the differences are significant in the paired bootstrap for every format (for example, Win32 tuned GT-Former challenge detection at 1% FPR: −12.0 points, CI −13.5 to −10.6). Optuna tuning narrows but does not close the gap: on Win32 it lifts GT-Former from 87.8% to 88.5% TPR at 0.1% FPR and from 33.2% to 41.5% challenge detection at 0.1% FPR (seed 0), the latter above the 40.5% of LightGBM.

Hybrid rank fusion: GT-Former is complementary to gradient boosting

A rank average of GT-Former and LightGBM scores is better than LightGBM alone on Win32 for every metric in Table 3, and the gains grow with the tuned GT-Former. With the default GT-Former, challenge detection at 1% FPR rises from 63.2% to 67.2% (+4.0 points, 95% CI +3.1 to +4.9) and novel-family detection from 89.1% to 91.2% (+2.1, CI +0.8 to +3.4). With the Optuna-tuned GT-Former the same fusion reaches 68.8% challenge detection (+5.6, CI +4.7 to +6.6), 91.8% novel-family detection (+2.7, CI +1.5 to +3.9) and 97.3% test TPR at 1% FPR (+0.4, CI +0.4 to +0.5), with TPR at 0.1% FPR rising from 90.0% to 91.0%. The tuned fusion is also significantly better than LightGBM on .NET challenge detection (79.3% to 80.8%, CI +0.1 to +3.1), on APK challenge detection (32.4% to

Table 3 Main comparison on the two largest EMBER2024 formats. Thresholds for challenge detection come from test benign files. Deep models use the plain objective, seed 0 (five-seed statistics in Section 4.1). Ch. = challenge-set detection; Nov. = novel-family TPR; LC = low-consensus TPR; @1% and @0.1% denote the false-positive budget. Best value per column and format in bold.

Model	ROC-AUC	TPR@1%	TPR@0.1%	Ch.@1%	Ch.@0.1%	Nov.@1%	LC@1%
Win32							
LightGBM (benchmark)	0.9982	0.9684	0.9002	0.6319	0.4047	0.8914	0.9095
MLP	0.9974	0.9540	0.8574	0.4874	0.3113	0.8811	0.8714
GT-Former (default)	0.9975	0.9598	0.8775	0.5011	0.3318	0.8935	0.8856
GT-Former (Optuna-tuned)	0.9976	0.9613	0.8852	0.5116	0.4149	0.8935	0.8900
FR-MoE	0.9973	0.9548	0.8535	0.4887	0.3234	0.8821	0.8741
ProtoCon-Net	0.9975	0.9555	0.8692	0.4859	0.3216	0.8842	0.8760
Fusion: GT-Former + LightGBM	0.9984	0.9708	0.9084	0.6719	0.4121	0.9121	0.9142
Fusion: tuned GT-Former + LightGBM	**0.9984**	**0.9727**	**0.9098**	**0.6881**	**0.4174**	**0.9183**	**0.9160**
Win64							
LightGBM (benchmark)	0.9988	0.9807	0.9198	0.5786	0.1130	0.9649	0.9616
MLP	0.9978	0.9735	0.8619	0.5086	0.1020	0.9518	0.9471
GT-Former (default)	0.9979	0.9733	0.8719	0.5258	0.1278	0.9535	0.9442
GT-Former (Optuna-tuned)	0.9981	0.9762	0.8739	0.5307	0.1118	0.9567	0.9490
Fusion: GT-Former + LightGBM	0.9987	0.9815	**0.9227**	**0.5872**	**0.1388**	0.9698	0.9621
Fusion: tuned GT-Former + LightGBM	**0.9988**	**0.9828**	0.9203	**0.5872**	0.1278	**0.9706**	**0.9626**



(a) challenge-set detection at 1% FPR

(b) challenge-set detection at 0.1% FPR

(c) test TPR at 0.1% FPR

(d) novel-family TPR at 1% FPR

Fig. 2 EMBER2024 operational metrics per format (thresholds from test benign files) for LightGBM, the deep models, the default and Optuna-tuned GT-Former, and the tuned fusion: (**a**) challenge-set detection at 1% FPR, (**b**) challenge-set detection at 0.1% FPR, (**c**) test TPR at 0.1% FPR, (**d**) novel-family TPR at 1% FPR.

36.3%, CI +0.8 to +7.4) and APK test TPR (+1.9, CI +1.6 to +2.2), and on Win64 test TPR (+0.2, CI +0.1 to +0.3); it is significantly worse only on the two smallest PE-like formats for test TPR (.NET −0.4, ELF −5.8), where the tuned transformer itself is unstable. On the unweighted mean over the six formats the tuned fusion has the best challenge detection at 1% and 0.1% FPR (0.5344 and 0.2750 versus 0.5130 and 0.2664 for LightGBM and the default fusion the best TPR at 1% FPR (0.9198), at an inference cost that Section Computational cost on matched hardware measures properly on matched hardware and which is considerably higher than the two-to-three-fold figure the submitted version reported from mismatched devices. Ensembling all three deep models with LightGBM is

Table 4 Ablation of the metadata-driven objectives on EMBER2024, unweighted mean over the six formats. Ch. = challenge-set detection; Nov. = novel-family TPR; LC = low-consensus TPR.

Variant	TPR@1%	TPR@0.1%	Ch.@1%	Ch.@0.1%	Nov.@1%	LC@1%
GT-Former plain	0.8769	0.6012	0.4629	0.2268	0.7317	0.7761
GT-Former full (CAM + cons. head + family GRL)	0.8187	0.4577	0.4395	0.2131	0.6860	0.7101
− CAM	0.8648	0.5793	0.4710	0.2155	0.7321	0.7553
− consensus head	0.7598	0.3438	0.4270	0.2348	0.6557	0.6841
− family GRL	0.8635	0.6295	0.4732	0.2263	0.6873	0.7663
MLP plain	0.8814	0.6821	0.4795	0.2182	0.7430	0.7857
MLP full	0.7406	0.3062	0.4491	0.2160	0.6697	0.7118
FR-MoE plain	0.8641	0.6633	0.4719	0.2430	0.7320	0.7685
FR-MoE full	0.8049	0.3526	0.4680	0.1051	0.7306	0.7389
ProtoCon-Net plain	0.8755	0.7049	0.4619	0.2308	0.7225	0.7774
ProtoCon-Net full	0.7413	0.3068	0.4277	0.1870	0.6701	0.6854
ProtoCon-Net − SupCon	0.7493	0.3521	0.4124	0.2099	0.7180	0.6956

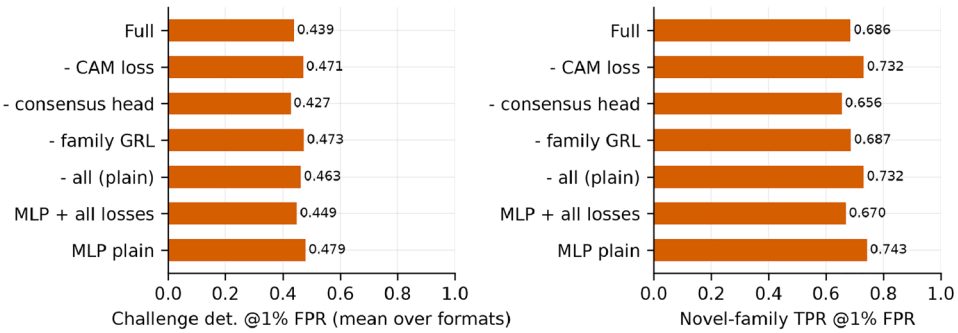


Fig. 3 GT-Former component ablation (mean over the six EMBER2024 formats).

worse than the two-model ensemble on Win32 (challenge detection at 1% FPR 52.4%), because the MLP and FR-MoE errors are highly correlated with each other and the average drowns the tree model.

Ablation of the metadata-driven objectives

Table 4 and Fig. 3 report the ablation of the three metadata-driven terms of Eq. (3) as unweighted means over the six formats. For every architecture the full objective lowers TPR at 0.1% FPR by 14–40 points and does not raise challenge detection. The validation curves tell the same story: GT-Former reaches 0.9964 validation AUC with the plain objective, 0.9963 without the family head, 0.9952 without CAM, 0.9947 without the consensus head and 0.9941 with everything switched on. Removing the family-adversarial head recovers the loss at 1% FPR and more than recovers it at 0.1% FPR, where the remaining pair of objectives reaches 0.6295 against 0.6012 for the plain model, so gradient reversal is the main culprit: forcing the encoder to forget family identity also removes information that separates malware from benign files. The novel-family stratum does not benefit either, reaching 0.6873 once the reversal head is removed against 0.7321 when CAM is the term removed instead. Because each row of Table 4 removes one term from the full objective, these numbers bound the marginal effect of a term *in the presence of the other two* and cannot isolate it; Section Activating each objective on its own reports the complementary add-one-in runs. CAM shifts the decision boundary for low-consensus malware, but the challenge files are not the low-consensus *training* files: CAM lowers TPR at strict FPR without improving challenge detection. The tree analogues behave the same way: consensus weights, family-balanced weights and their inverse control are all within ± 2 points of the benchmark on every metric.

Drift over the twelve test weeks

Figure 4 plots TPR at 1% FPR per test week. It is flat on Win32, Win64 and .NET for all models (Win32 LightGBM: 0.973 in week 52 and 0.962 in week 63). The small formats drift: on APK all models lose 6–7 points between weeks 52 and 60 in lock-step (LightGBM 0.823 to 0.758), and on PDF the MLP and FR-MoE collapse to 0.63 and 0.49 in weeks 59–62 while LightGBM (0.91) and GT-Former (0.80) hold up. The group-token architecture is therefore markedly more robust to the PDF benign-distribution shift than the flat encoders, though still less robust than the tree model.

Seed variability

Over five seeds the deep models are stable on the large formats and unstable on the smallest. On Win32 the default GT-Former reaches TPR at 0.1% FPR of 0.879 ± 0.003, challenge detection at 1% FPR of 0.495 ± 0.004 and novel-family TPR of 0.891 ± 0.004; the tuned GT-Former reaches 0.881 ± 0.004, 0.523 ± 0.031 and 0.892 ± 0.003; the MLP 0.859 ± 0.001, 0.487 ± 0.003 and 0.888 ± 0.006; and FR-MoE 0.857 ± 0.004, 0.488 ± 0.003 and 0.886 ± 0.004. On Win64 the tuned GT-Former reaches challenge detection of 0.541 ± 0.009 versus 0.531 ± 0.005 for the default. On .NET the tuned GT-Former reaches challenge detection of 0.800 ± 0.011 (LightGBM 0.793) and novel-family TPR of 0.844 ± 0.016. On ELF the TPR at 0.1% FPR of GT-Former is 0.14 ± 0.14 (default) and 0.13 ± 0.14 (tuned) across seeds, so the transformer is unreliable on 26,000 training files; the MLP is the most stable deep model there (0.53 ± 0.08). Any claim about ELF or PDF from a single deep run should be treated with suspicion. The instability is not a class-imbalance effect: the ELF test set holds 2,989 benign and 3,000 malicious files and the PDF test set 6,000 of each, so both are balanced. It is a small-sample effect on the benign side of a strict budget. At 0.1% FPR the operating threshold on ELF is set by the three highest-scoring of 2,989 benign files, so the budget permits two false positives and the threshold is the third-highest benign score; on PDF it permits six. A single benign outlier that one seed scores high and another scores low therefore moves the threshold, and with it TPR at 0.1% FPR, by a large amount, which is why the deep models, whose benign score tails vary more between seeds than the tree's, show spreads of ± 0.14 on ELF while LightGBM is deterministic. The same mechanism explains why the validation-calibrated budget

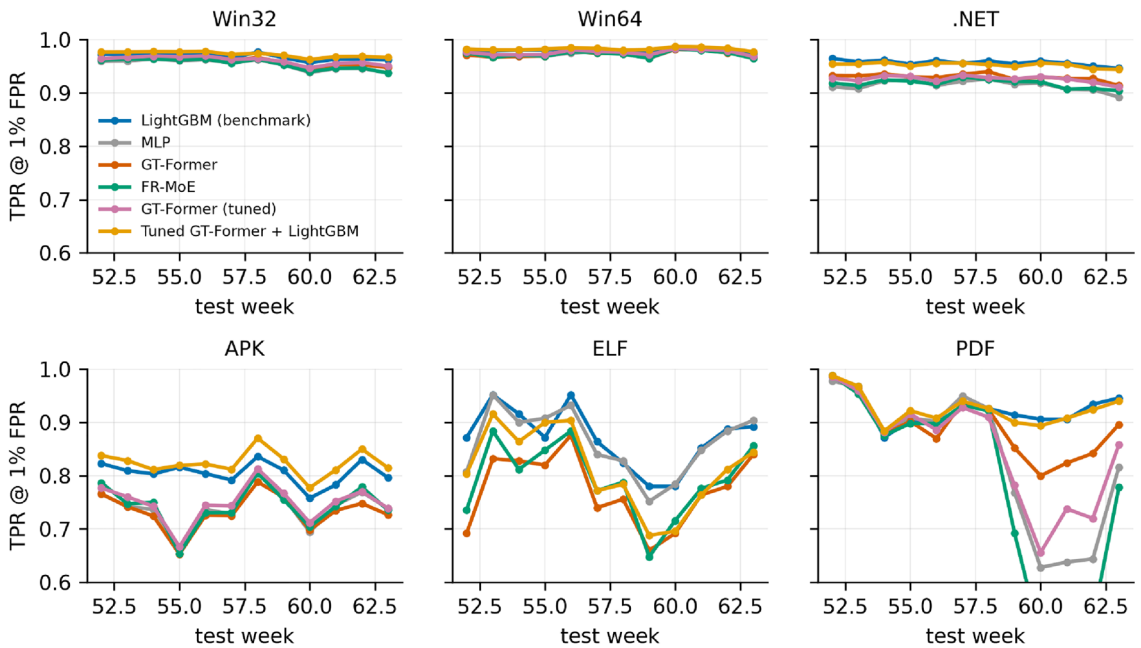


Fig. 4 EMBER2024: TPR at 1% FPR per test week (weeks 52–63) and format.

roughly doubles on these formats (Section 4.1): the validation benign sets are smaller still, and the threshold they set is correspondingly noisier.

Calibrated thresholds

With the threshold chosen on validation benign files to give 1% FPR, the realised FPR on the future test set is 0.74–1.30% on the PE formats for all models, 1.8–3.3% on ELF and 1.5–2.3% on PDF, so benign drift on the small formats roughly doubles the false-positive budget. Challenge detection at the validation-calibrated threshold is 60.2% for LightGBM and 60.9% for the fusion on Win32.

Computational cost

Training LightGBM on all formats takes 43 minutes; the default GT-Former trains in 9 minutes and the tuned one in 22 minutes on a laptop GPU; the Optuna study cost 20 proxy trials of about one minute each. Inference cost is measured on matched hardware in Section Computational cost on matched hardware.

Classical metrics, confusion matrices and curves

For readers who prefer the conventional view, Tables 5–13 report the derived classical metrics (accuracy, balanced accuracy, precision, recall, specificity, FPR, F1 and Matthews correlation coefficient) for the reference LightGBM, the MLP, the Optuna-tuned GT-Former and the fusion, on every benchmark and format. The operating point is the deployment-realistic one: the score threshold that yields 1% FPR on the *validation* benign files, applied unchanged to the future test data, so that the realised FPR is what a deployment would see. Figures 5, 6, 7, 8, 9, 10 give, for each EMBER2024 format, the confusion matrices at this threshold and the ROC curves (logarithmic FPR axis) and precision–recall curves on the test set and on the challenge set; the vertical dotted lines in the ROC panels mark the 1% and 0.1% FPR operating points. Three observations complement the operational analysis. First, at the calibrated 1% threshold the realised FPR stays at 0.8–0.9% on Win32 and Win64 and at 1.2–1.3% on .NET for all models, so the calibration transfers to the future test weeks on the large PE formats, whereas on the small formats it drifts to 1.7–3.3%. Second, accuracy and F1 hide the differences that matter: on Win32 the fusion improves accuracy over LightGBM by only 0.17 points (0.9786 to 0.9803) and MCC by 0.003 (0.9575 to 0.9608), yet it recovers 494 additional malware files (5,534 versus 6,028 false negatives) with 109 fewer false positives; on the challenge set mixed with test benign files the F1 of the fusion is 0.584 versus 0.568 and its MCC 0.577 versus 0.561. Third, under strong drift the classical metrics collapse for every model: on LAMDA FAR the recall at the calibrated threshold is 0.23 for LightGBM and 0.25 for the MLP with precision above 0.97, so the detectors are conservative rather than noisy, and the rank fusion keeps the FPR at 0.12% while lifting the AUC to 0.916.

LAMDA

Reproduction of the benchmark protocol

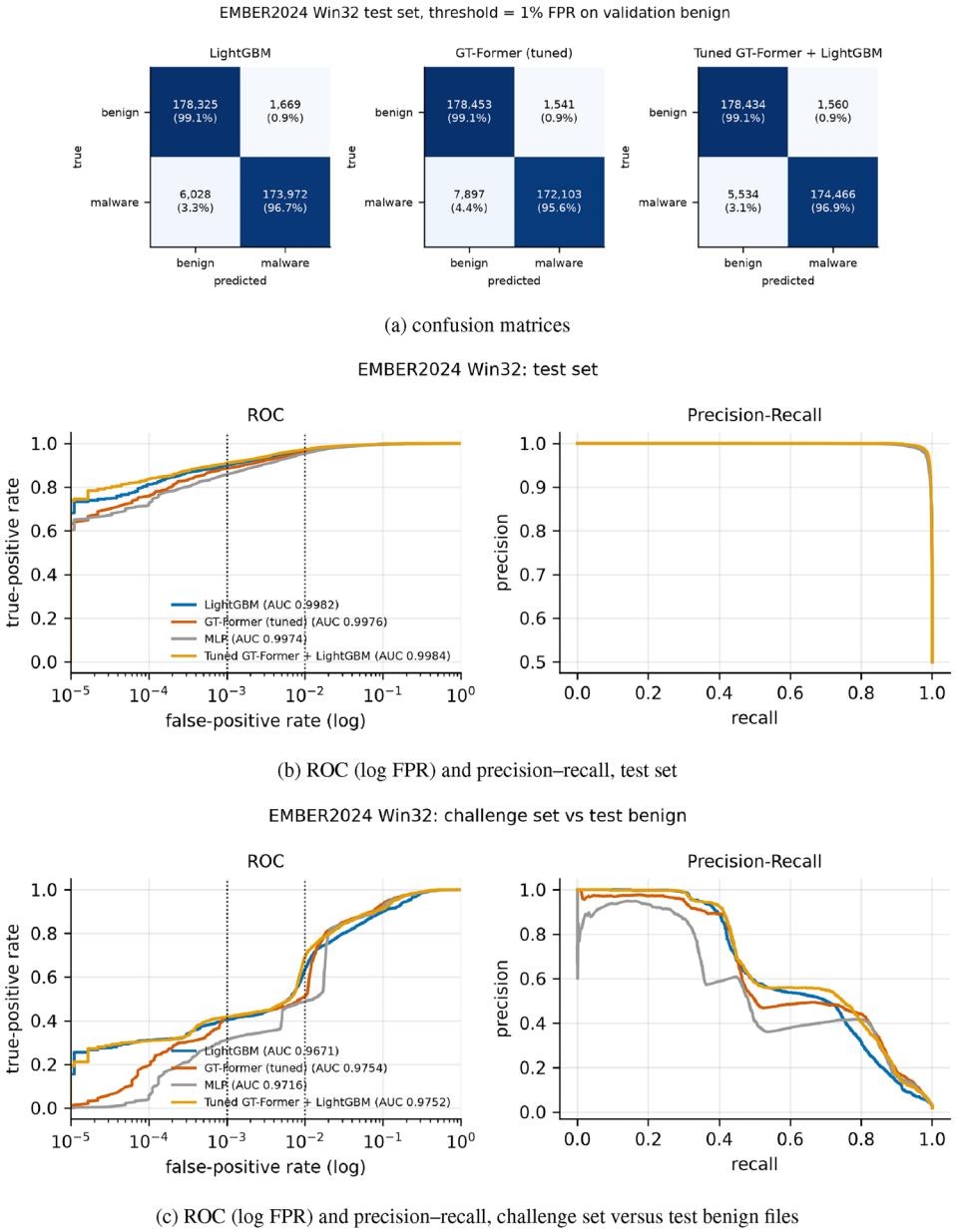
Our LightGBM reproduces the IID numbers of the LAMDA paper exactly (F1 97.47% versus 97.49 ± 0.17; FNR 2.2%; FPR 2.4%) and lands within its reported variance on the drift regions (NEAR F1 73.1% versus 59.5 ± 28.2; FAR 58.7% versus 47.2 ± 27.3; the paper's standard deviations are large because they average over years). The AUC picture is the same: 0.9963 on IID, 0.830 on NEAR and 0.886 on FAR.

Table 5 EMBER2024 classical metrics at the validation-calibrated 1% FPR threshold. Fusion = rank fusion of the tuned GT-Former and LightGBM. The Win32 chall. rows mix the challenge files with the Win32 test benign files.

Subset	Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	FPR	F1	MCC	ROC-AUC	PR-AUC
Win32	LightGBM	0.9786	0.9786	0.9905	0.9665	0.9907	0.0093	0.9784	0.9575	0.9982	0.9983
Win32	MLP	0.9696	0.9696	0.9914	0.9474	0.9917	0.0083	0.9689	0.9401	0.9974	0.9976
Win32	GT-Former (tuned)	0.9738	0.9738	0.9911	0.9561	0.9914	0.0086	0.9733	0.9482	0.9976	0.9979
Win32	Fusion	0.9803	0.9803	0.9911	0.9693	0.9913	0.0087	0.9801	0.9608	0.9984	0.9985
Win32 chall.	LightGBM	0.9839	0.7963	0.5377	0.6019	0.9907	0.0093	0.5680	0.5607	0.9671	0.6441
Win32 chall.	MLP	0.9828	0.7362	0.5107	0.4806	0.9917	0.0083	0.4952	0.4867	0.9716	0.5590
Win32 chall.	GT-Former (tuned)	0.9827	0.7446	0.5102	0.4977	0.9914	0.0086	0.5038	0.4951	0.9754	0.6391
Win32 chall.	Fusion	0.9847	0.8016	0.5584	0.6118	0.9913	0.0087	0.5839	0.5767	0.9752	0.6697
Win64	LightGBM	0.9843	0.9843	0.9924	0.9759	0.9926	0.0074	0.9841	0.9686	0.9988	0.9989
Win64	MLP	0.9810	0.9810	0.9908	0.9711	0.9909	0.0091	0.9809	0.9623	0.9978	0.9981
Win64	GT-Former (tuned)	0.9826	0.9826	0.9906	0.9744	0.9907	0.0093	0.9824	0.9653	0.9981	0.9983
Win64	Fusion	0.9862	0.9862	0.9906	0.9817	0.9907	0.0093	0.9861	0.9725	0.9988	0.9990
.NET	LightGBM	0.9758	0.9758	0.9881	0.9633	0.9883	0.0117	0.9755	0.9519	0.9980	0.9980
.NET	MLP	0.9603	0.9604	0.9863	0.9337	0.9870	0.0130	0.9593	0.9220	0.9962	0.9962
.NET	GT-Former (tuned)	0.9658	0.9658	0.9865	0.9446	0.9871	0.0129	0.9651	0.9325	0.9966	0.9964
.NET	Fusion	0.9748	0.9748	0.9869	0.9623	0.9872	0.0128	0.9745	0.9498	0.9978	0.9979
APK	LightGBM	0.8900	0.8900	0.9894	0.7884	0.9915	0.0085	0.8775	0.7966	0.9842	0.9854
APK	MLP	0.8602	0.8602	0.9879	0.7295	0.9910	0.0090	0.8392	0.7465	0.9771	0.9790
APK	GT-Former (tuned)	0.8603	0.8603	0.9885	0.7291	0.9915	0.0085	0.8392	0.7468	0.9774	0.9795
APK	Fusion	0.8998	0.8998	0.9901	0.8077	0.9919	0.0081	0.8896	0.8135	0.9855	0.9868
ELF	LightGBM	0.9566	0.9566	0.9810	0.9313	0.9819	0.0181	0.9555	0.9144	0.9845	0.9874
ELF	MLP	0.9486	0.9486	0.9661	0.9300	0.9672	0.0328	0.9477	0.8978	0.9881	0.9893
ELF	GT-Former (tuned)	0.9546	0.9546	0.9766	0.9317	0.9776	0.0224	0.9536	0.9101	0.9816	0.9757
ELF	Fusion	0.9594	0.9595	0.9745	0.9437	0.9752	0.0248	0.9588	0.9193	0.9859	0.9860
PDF	LightGBM	0.9608	0.9608	0.9764	0.9445	0.9772	0.0228	0.9602	0.9222	0.9906	0.9927
PDF	MLP	0.9472	0.9472	0.9817	0.9113	0.9830	0.0170	0.9452	0.8966	0.9834	0.9871
PDF	GT-Former (tuned)	0.9530	0.9530	0.9740	0.9308	0.9752	0.0248	0.9519	0.9069	0.9752	0.9822
PDF	Fusion	0.9639	0.9639	0.9761	0.9512	0.9767	0.0233	0.9635	0.9281	0.9865	0.9898

Operational metrics under drift

Table 6 reports the single models and the rank ensembles per region, and Figs. 14 and 15 show the year-by-year and region-level views. Three findings mirror EMBER2024 and one does not. First, in distribution and in the near future LightGBM is the best single model (NEAR TPR at 1% FPR 52.1% versus 39.9% for the MLP, CI for the difference −12.6 to −12.0). Second, the evasive strata are where everything fails: singleton and low-consensus malware are detected at 30% or less by every single model on NEAR and FAR. Third, the metadata-driven objectives hurt again: the full GT-Former loses 5.7 points of NEAR TPR at 1% FPR relative to the plain model (0.191 versus 0.248) and 6.4 points on FAR, and the full MLP loses 3.6 points on NEAR and 8.0 on FAR. The finding that does *not* transfer is the ranking of deep models. On LAMDA the flat MLP is the most drift-robust single model on the FAR years (TPR at 1% FPR 49.1% versus 40.5% for LightGBM, +8.6 points, CI +8.4 to +8.8; singleton detection 48.9% versus 34.6%), whereas GT-Former is the weakest even after Optuna tuning (FAR TPR at 1% FPR 0.28 ± 0.03 default and 0.29 ± 0.05 tuned over five seeds; MLP 0.49 ± 0.03; LightGBM has no seed variance at this configuration). Tuning raised the validation AUC of the transformer from 0.9941 to 0.9947 and its NEAR TPR from 24.8% to 29.3%, but the gap to the MLP remains 20 points. Tokenising 4,561 binary indicator features by Drebin category discards the fine-grained co-occurrence information that the flat encoder exploits, whereas on EMBER version-3 features the groups are semantically coherent numeric blocks. The choice of architecture must follow the semantics of the feature vector.



(a) confusion matrices

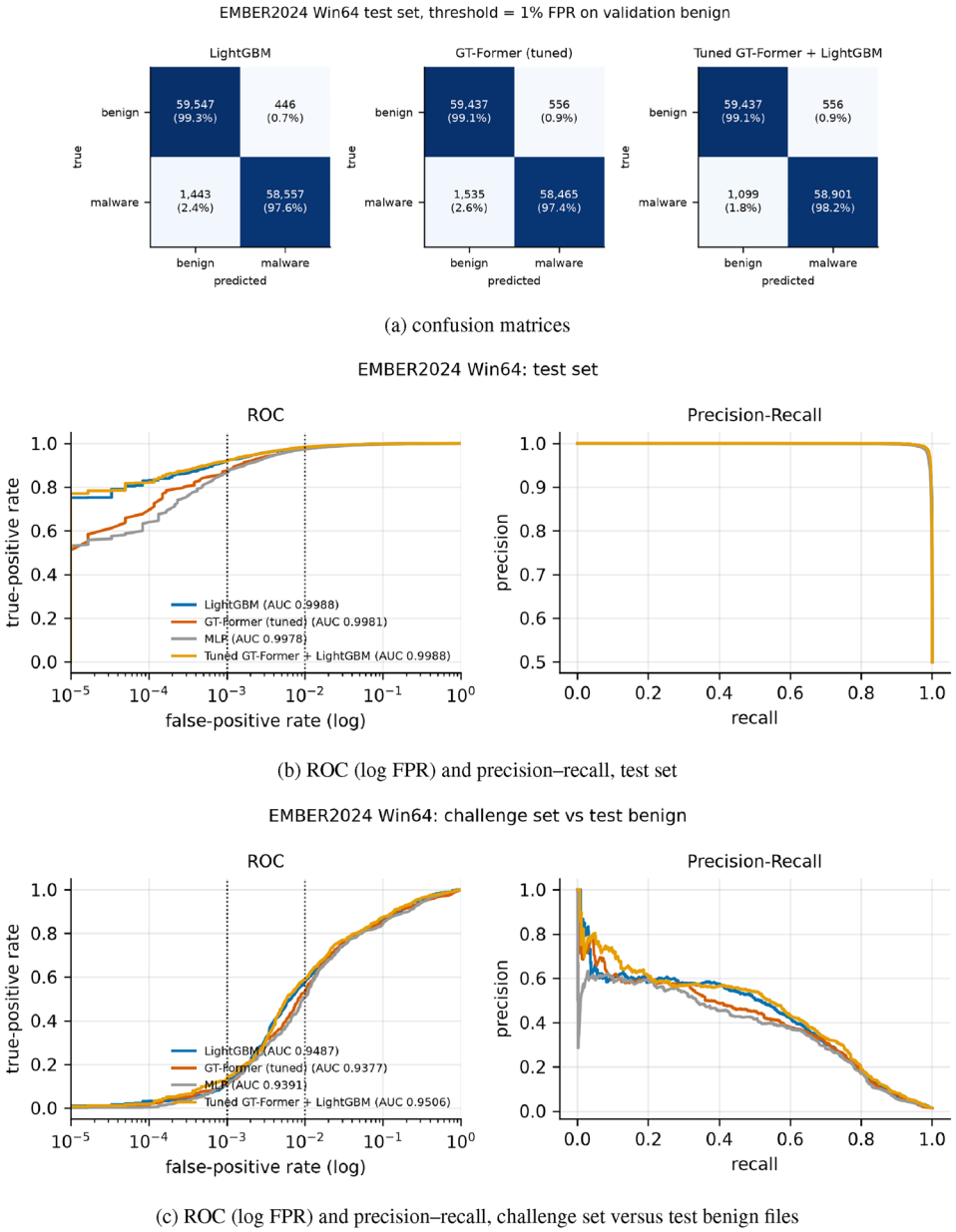
(b) ROC (log FPR) and precision–recall, test set

(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

Fig. 5 EMBER2024 Win32: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

Hybrid rank fusion under strong drift

A rank average of the MLP and LightGBM is the best model on the FAR region on every *operational* metric, though not on ROC-AUC, where the MLP alone is marginally higher (0.9184 versus 0.9163): TPR at 1% FPR 54.4% versus 40.5% for LightGBM (+13.8 points, CI +13.7 to +14.0), novel-family TPR 35.9% versus 29.5% (+6.4 , CI +5.9 to +6.9), singleton TPR 53.3% versus 34.6% (+18.7, CI +18.4 to +18.9), low-consensus TPR 40.4% versus 26.3%, and TPR at 0.1% FPR 21.1% versus 12.9%. On NEAR the ensemble is slightly below LightGBM (48.4% versus 52.1%, −3.7 points) and on IID slightly above (94.4% versus 93.6%). Adding the tuned GT-Former to the ensemble reduces the FAR gain to +7.3 points (47.8% TPR), and a tuned-GT-Former/LightGBM pair is below LightGBM (37.4%): on Drebin features the transformer adds noise, not signal. Figures 11, 12, 13 show the confusion matrices and the ROC and precision–recall curves per region at the validation-calibrated threshold, and Table 7 the corresponding classical metrics. The practical recipe is therefore the same on both benchmarks: keep



(a) confusion matrices

(b) ROC (log FPR) and precision–recall, test set

(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

Fig. 6 EMBER2024 Win64: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

the gradient-boosted benchmark, add one deep model with a different inductive bias, and fuse by rank; the deep partner should be GT-Former on EMBER version-3 features and a flat MLP on Drebin-style binary features.

Computational cost on matched hardware

The submitted version timed LightGBM on the CPU and the deep models on the GPU and concluded that the fusion costs two to three times more than the tree. That comparison was not like for like. Table 8 re-measures every model on the same machine (Intel six-core, twelve logical threads, 67.9 GB RAM, one RTX A4000 Laptop GPU with 8 GB) on the same 359,994 Win32 test vectors, separating batch throughput (batch size 2,048, 20,000 files) from single-file latency (2,000 sequential calls), with CUDA synchronisation before every GPU measurement so that asynchronous launches cannot flatter the result.

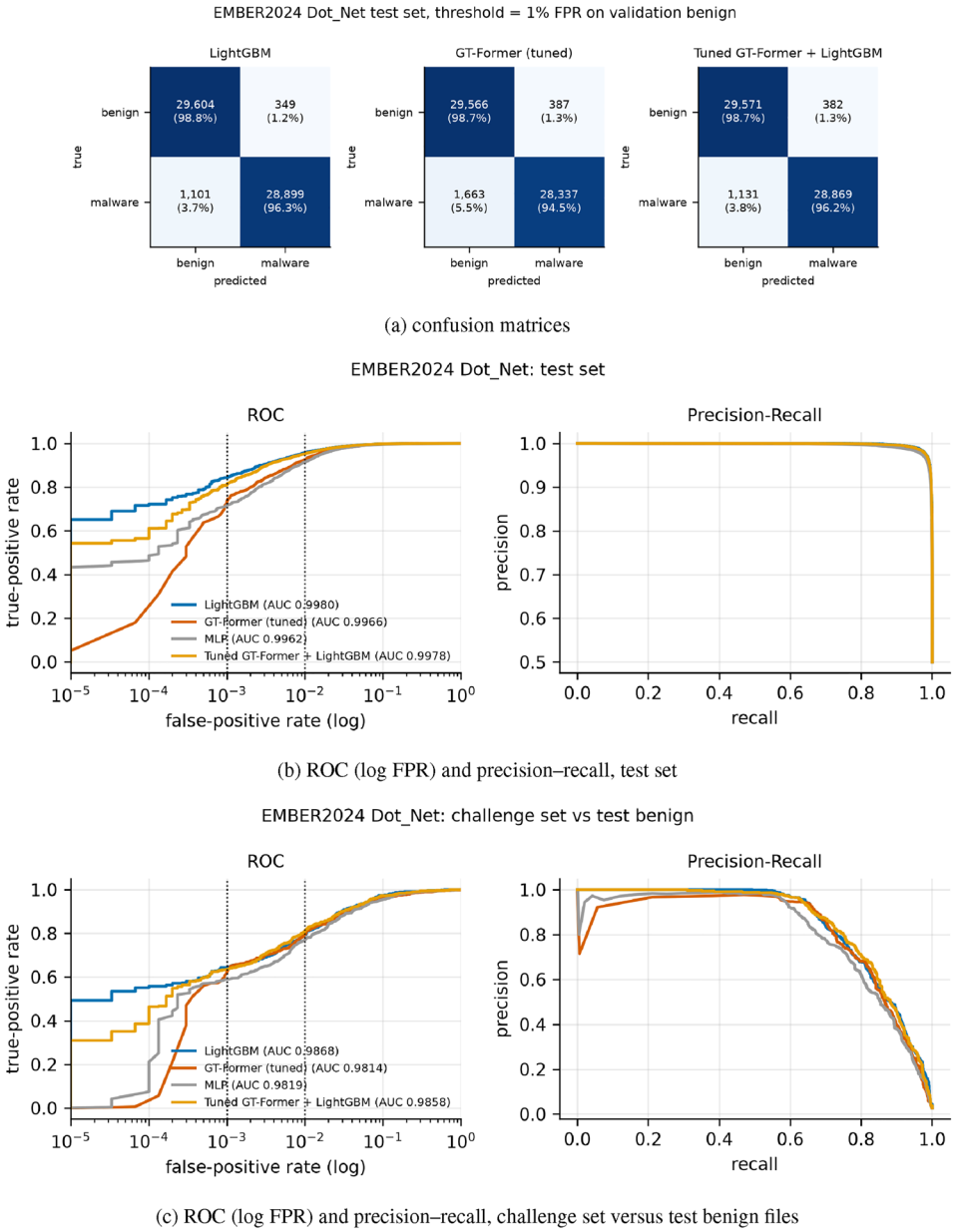


Fig. 7 EMBER2024 .NET: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

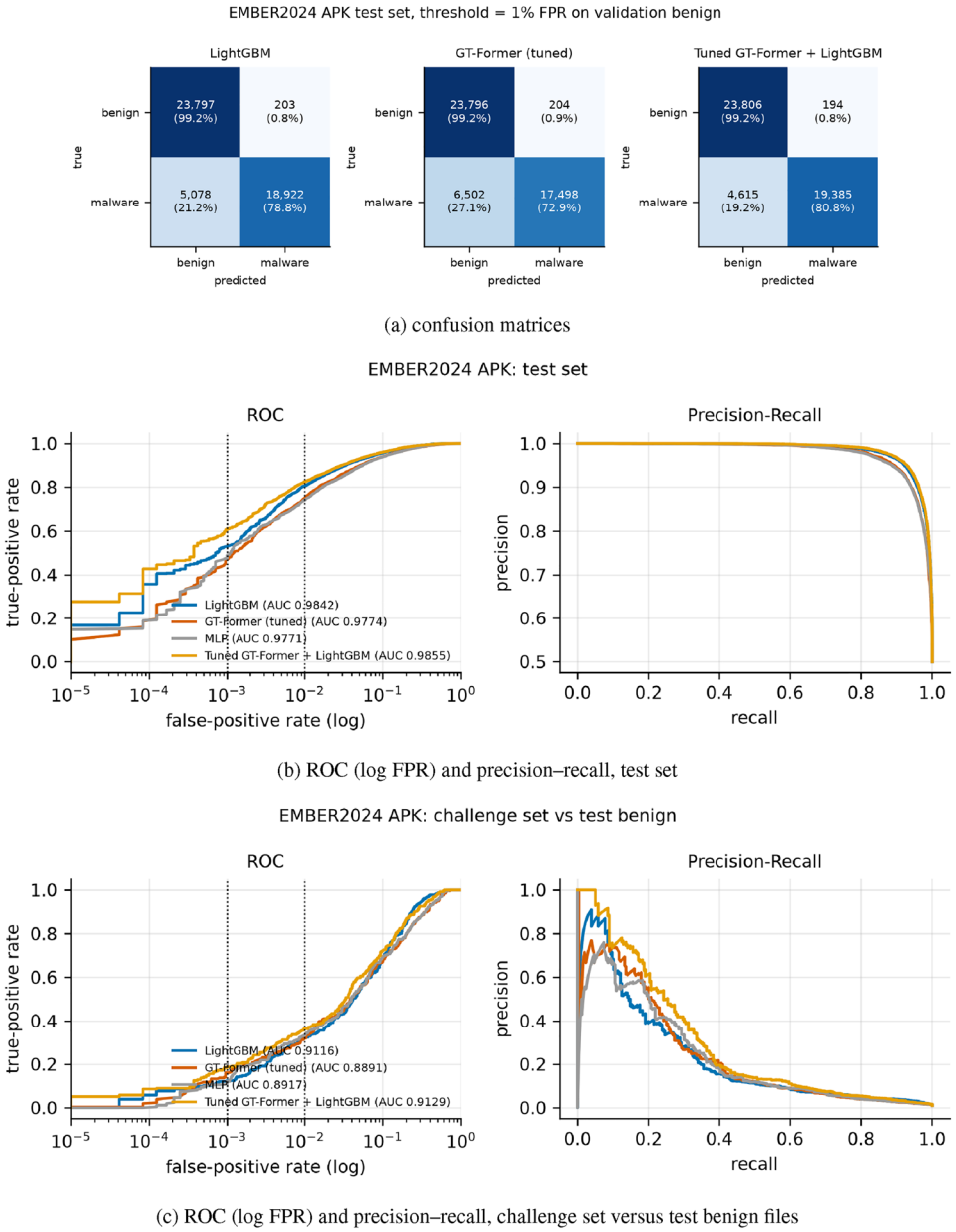
(a) confusion matrices

(b) ROC (log FPR) and precision–recall, test set

(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

Three corrections follow. First, the tree is not marginally cheaper but decisively cheaper: on twelve CPU threads LightGBM scores 117,253 files per second against 18,017 for the tuned transformer on a dedicated GPU, a factor of 6.5, and 1,055 on the same CPU, a factor of 111. Per file scored on its own the gap is 147 μs against 3,223 μs, a factor of 22 in favour of the tree even when the transformer has a GPU and the tree does not. The "two to three times" of the submitted version was an artefact of comparing GPU batch throughput with CPU batch throughput, and we withdraw it.

Second, throughput and latency are not interchangeable, and the difference is large. LightGBM scores 8.5 μs per file in a batch of 2,048 but 147 μs when a single file is scored alone, because thread startup dominates a batch of one; adding threads makes latency *worse* (237.6 μs at twelve threads against 147.4 μs at one) while improving throughput sevenfold. A deployment that scans files as they arrive pays the latency figure, and one that sweeps a corpus pays the throughput figure. We report both throughout.



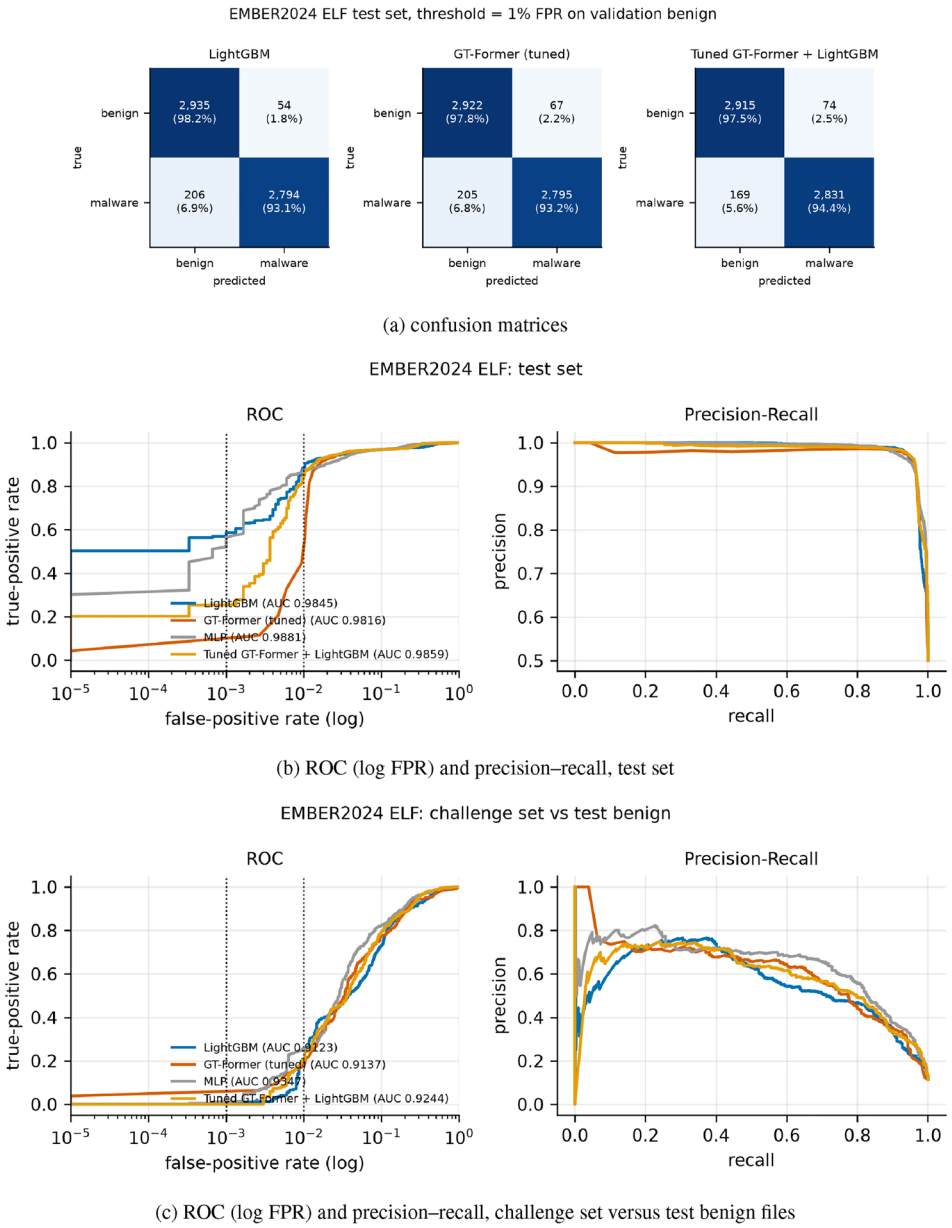
(a) confusion matrices

(b) ROC (log FPR) and precision–recall, test set

(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

Fig. 8 EMBER2024 APK: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

Third, and most important operationally, feature extraction dominates everything. Vectorising the 1,560,000 Win32 training files with `thrember` took sixteen minutes of wall clock on twelve logical cores, about 615 μs per file, which is four times the LightGBM latency and of the same order as the transformer's. Any end-to-end static detector is therefore bounded by parsing, not by classification, and the choice between these models changes total cost by a factor well under two once extraction is included. The rank fusion itself is free: two lookups into a stored percentile map cost 0.288 μs per member per file, five orders of magnitude below the models they combine. The case for the fusion rests on detection quality, not on cost, and the case against the transformer as a standalone deployment is stronger than we previously reported.



(a) confusion matrices

(b) ROC (log FPR) and precision–recall, test set

(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

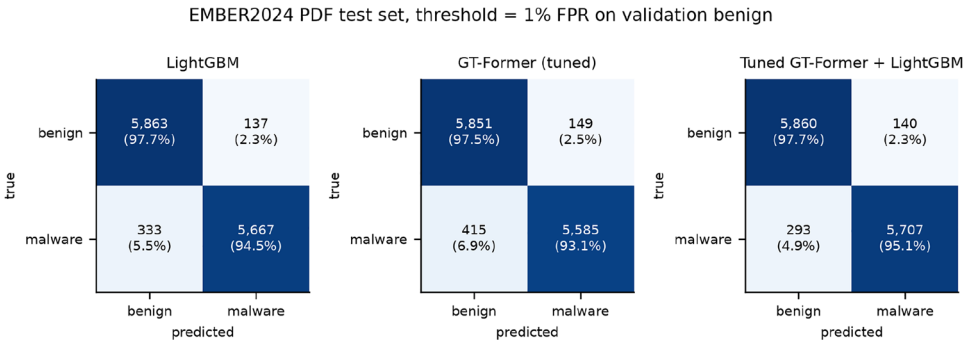
Fig. 9 EMBER2024 ELF: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

Activating each objective on its own

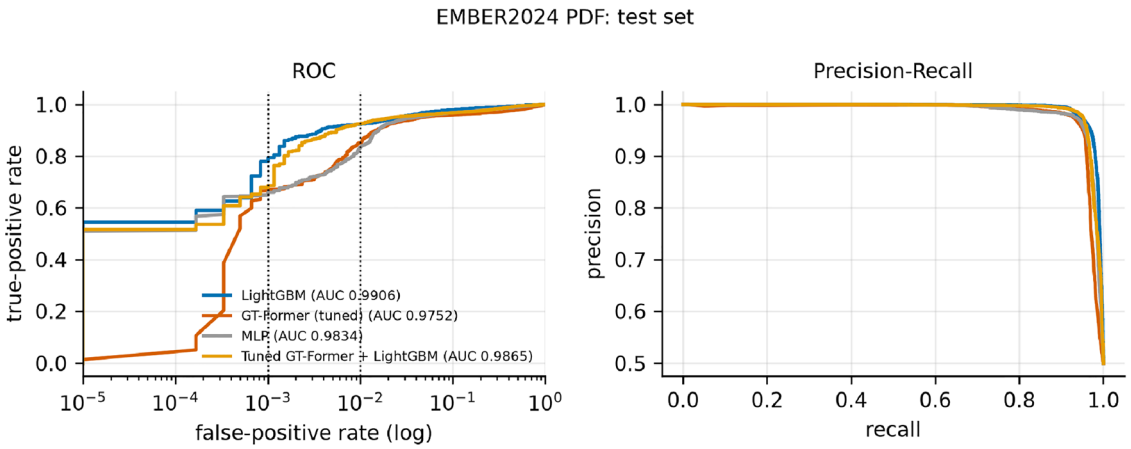
Table 4 removes one term at a time from the full objective, which bounds the marginal effect of a term *in the presence of the other two* but cannot isolate it. Following the reviewers' request we ran the complementary experiment on LAMDA, activating exactly one objective on top of the plain binary cross-entropy baseline, with five seeds for every variant including the full model. Table 9 reports the result, and it does not support the conclusion we drew in the submitted version.

Three conclusions follow, and the first two correct the submitted manuscript. First, *no objective activated on its own is distinguishable from the plain model*. Every individual variant lies within one standard deviation of the baseline on every column; the largest apparent gain, +5.5 points of NEAR TPR at 1% FPR for the gradient-reversal head, carries a spread of ±4.18 and is accompanied by a *loss* at the stricter budget (6.88 ± 2.10 against

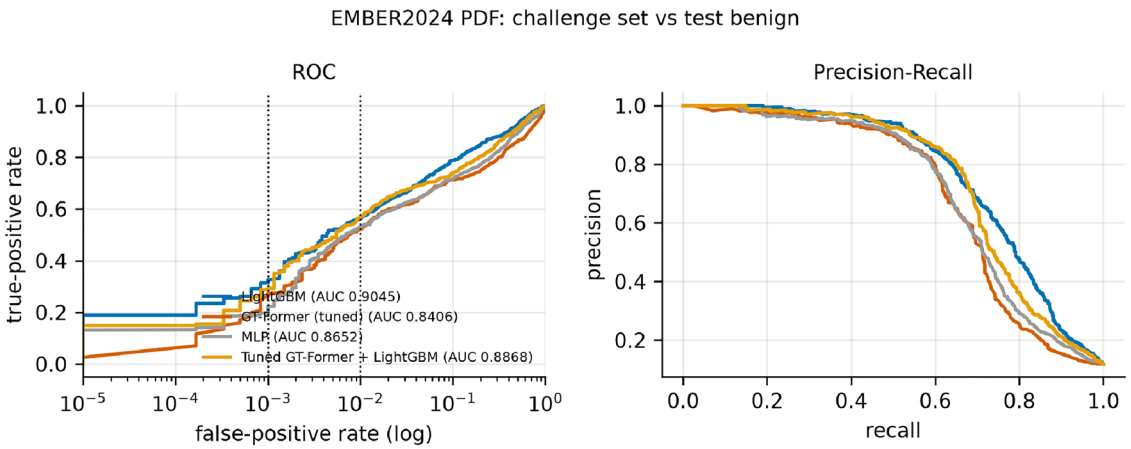
https://doi.org/10.1038/s41598-026-75034-w



(a) confusion matrices



(b) ROC (log FPR) and precision–recall, test set



(c) ROC (log FPR) and precision–recall, challenge set versus test benign files

Fig. 10 EMBER2024 PDF: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves on the test set, (**c**) the same curves on the challenge set. Dotted lines mark the 1% and 0.1% FPR operating points.

Table 6 LAMDA: single models (seed 0) and rank ensembles. TPR at 1% FPR uses the threshold that gives 1% FPR on the benign files of the same region; strata as in Section Evaluation protocol (Known = known-family TPR; Nov. = novel-family TPR, only 13 such files exist on IID; Single. = singleton TPR; LC = low-consensus TPR). F1 is at threshold 0.5 and is only meaningful for single models. Best value per column in bold.

Model	Region	ROC-AUC	F1@0.5	TPR@1%	TPR@0.1%	Known@1%	Nov.@1%	Single.@1%	LC@1%
LightGBM	IID	0.9963	0.9747	0.9364	0.6824	0.9469	0.1538	0.7900	0.8957
LightGBM	NEAR	0.8296	0.7308	0.5213	0.2678	0.6080	0.3097	0.3057	0.2851
LightGBM	FAR	0.8855	0.5869	0.4053	0.1287	0.5808	0.2950	0.3460	0.2633
MLP	IID	0.9954	0.9697	0.9272	0.6420	0.9400	0.1538	0.7425	0.8683
MLP	NEAR	0.8431	0.6746	0.3986	0.1131	0.4803	0.1979	0.1953	0.1609
MLP	FAR	**0.9184**	0.5855	**0.4914**	**0.1777**	0.5804	0.2756	**0.4890**	**0.3614**
GT-Former (default)	IID	0.9947	0.9682	0.9269	0.5860	0.9376	0.0769	0.7800	0.8787
GT-Former (default)	NEAR	0.8381	0.6558	0.2480	0.1159	0.3139	0.0899	0.0834	0.0623
GT-Former (default)	FAR	0.8686	0.3536	0.2304	0.0752	0.5398	0.1887	0.0995	0.0881
GT-Former (tuned)	IID	0.9953	0.9709	0.9271	0.6665	0.9380	0.1538	0.7775	0.8796
GT-Former (tuned)	NEAR	0.8554	0.6595	0.2927	0.0742	0.3610	0.1137	0.1044	0.0855
GT-Former (tuned)	FAR	0.8708	0.3521	0.2449	0.0749	0.5486	0.1985	0.1178	0.1065
ProtoCon-Net	IID	0.9951	0.9714	0.9110	0.5907	0.9242	0.1538	0.7200	0.8530
ProtoCon-Net	NEAR	0.8356	0.6663	0.3685	0.1203	0.4472	0.1850	0.1701	0.1393
ProtoCon-Net	FAR	0.9109	0.5930	0.4804	0.1749	0.5629	0.2482	0.4837	0.3589
Fusion: MLP + LightGBM	IID	0.9964	–	**0.9436**	0.6793	0.9533	0.1538	0.8100	0.9011
Fusion: MLP + LightGBM	NEAR	0.8417	–	0.4844	**0.2825**	0.5814	0.2761	0.2349	0.2198
Fusion: MLP + LightGBM	FAR	0.9163	–	**0.5436**	**0.2106**	**0.6400**	**0.3586**	**0.5326**	**0.4041**

10.08 ± 2.26). We therefore withdraw the claim that each objective individually lowers strict-FPR performance, and in particular the attribution of the loss to the family-adversarial head.

Second, the one effect that exceeds the seed spread appears only when all three terms are combined, and only at the strictest operating point: NEAR TPR at 0.1% FPR falls from 10.08 ± 2.26 to 4.43 ± 3.67, and the FAR singleton and novel-family strata fall by three to four points. The harm is therefore a property of the *combination*, not of any single term, which is consistent with the leave-one-out pattern in Table 4, where removing the gradient-reversal head and keeping the other two recovers and exceeds the plain model at 0.1% FPR (0.6295 against 0.6012).

Third, and most useful to anyone extending this line of work, the seed spread on LAMDA is large enough to swamp the effects being measured. A single seed of the full objective gives 19.39% NEAR TPR at 1% FPR, while five seeds give 30.83 ± 6.34%; the same variant would support opposite conclusions depending on which seed was run. Our own submitted conclusion rested on single-seed runs and did not survive replication. Table 4 on EMBER2024 is likewise single-seed and should be read as indicative rather than conclusive; we report it unchanged for completeness but no longer draw per-objective conclusions from it. Ablation studies on drifting malware benchmarks need seed replication before any claim about an individual loss term, and this is as much a finding of the paper as any of the detection numbers.

Sensitivity of the fusion to its weighting

The fusion of Section 4.1 averages the two percentile scores equally. Because an equal weight is a choice rather than a result, Table 10 sweeps it: the fused score is $(1 - w)\,\hat{F}_{LightGBM}(s) + w\,\hat{F}_{GT\text{-}Former}(s)$ with w from 0 (the tree alone) to 1 (the transformer alone), in steps of 0.1, over the five tuned seeds.

Three things follow. First, the equal weight is close to optimal but not optimal: challenge detection peaks at $w = 0.3$ with 68.73 ± 0.67 against 68.20 ± 0.49 at $w = 0.5$, so the best mixture leans slightly towards the tree. Second, and more important for practice, the optimum is flat. Any weight between 0.2 and 0.5 yields 68–69% challenge detection, so the gain is not an artefact of a tuned mixing constant and needs no per-deployment calibration; we keep $w = 0.5$ in the rest of the paper because it is the parameter-free choice and is within one standard deviation

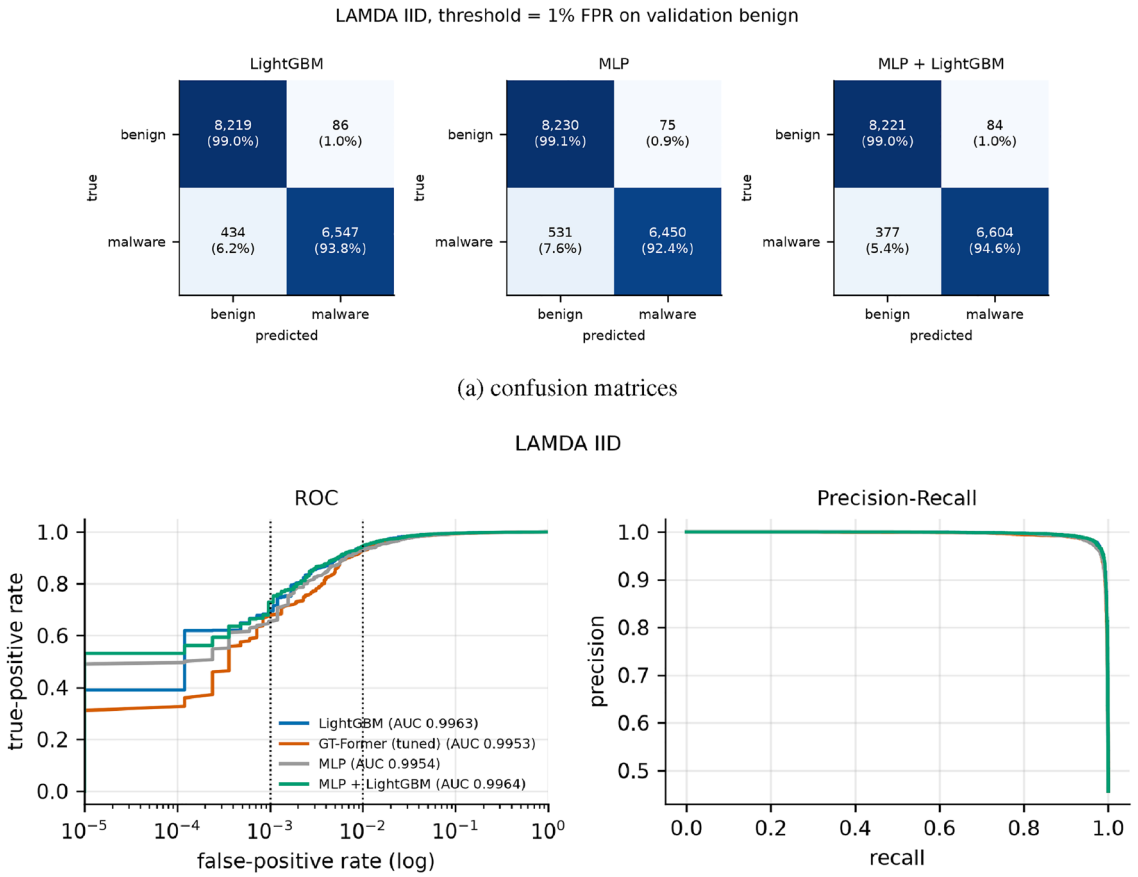


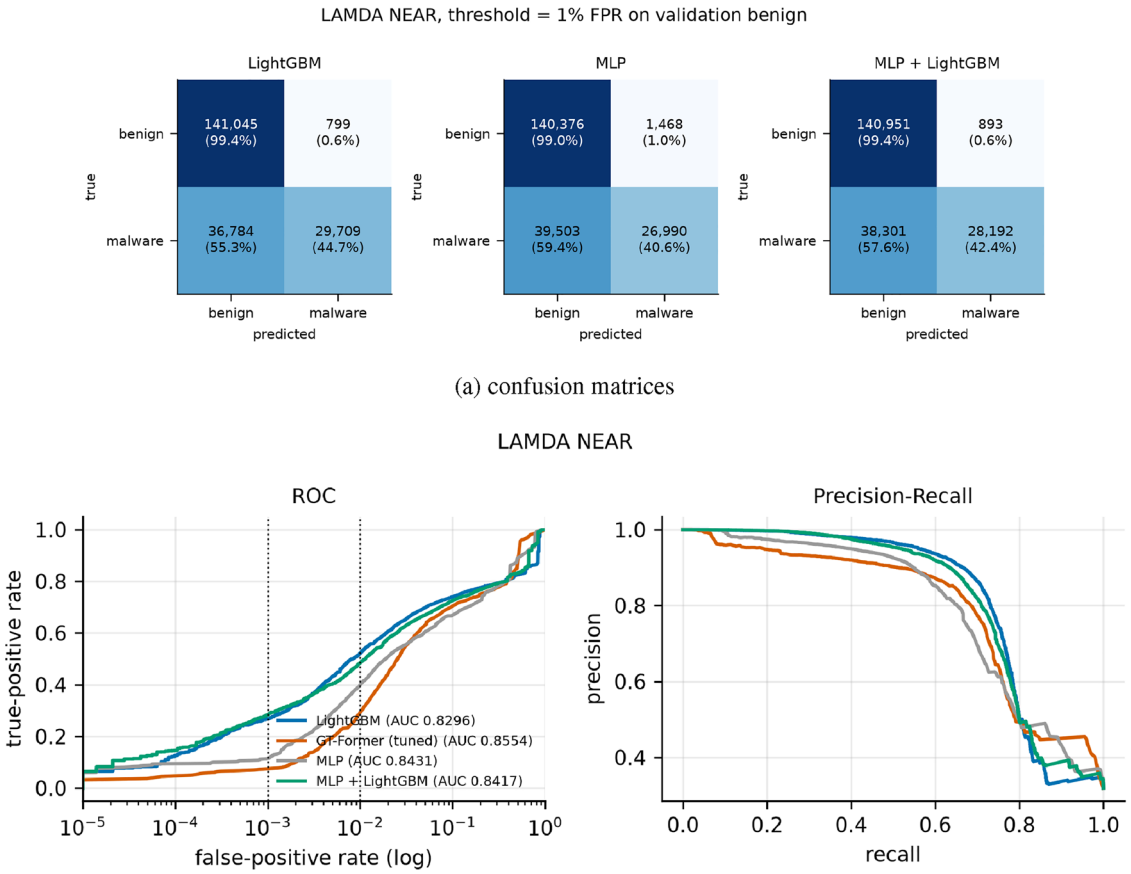
Fig. 11 LAMDA IID region: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves. Dotted lines mark the 1% and 0.1% FPR operating points.

of the best. Third, the seed variance is strongly asymmetric: it is negligible at low w and reaches ± 4.15 points at $w = 0.9$, which identifies the transformer rather than the tree as the unstable partner and is consistent with the seed spreads reported in Section Results. Weighting the fusion towards the deep model therefore buys nothing and costs reproducibility.

Retrospective and deployment operating points

Every fixed-FPR number in this paper can be read in two ways, and the difference is large enough that we separate them explicitly. A *retrospective* operating point takes the threshold from the benign files of the evaluation cohort itself, which is what prior work on these benchmarks reports and what makes our numbers comparable to it. A *deployment* operating point takes the threshold from validation data collected before the test period and applies it unchanged, so the realised false-positive rate is whatever drift makes it. Table 11 gives both for the two fusions that carry our main claims.

On EMBER2024 the advantage survives the transfer, shrinking from $+5.6$ to $+0.9$ points of challenge detection, because twelve test weeks contain little drift and the calibration holds (realised FPR 0.87–0.93% against a 1% budget). On LAMDA it does not. Over 2018–2025 the score distributions of both models move so far that a threshold fixed on 2013–2014 validation data yields 0.35% FPR for LightGBM and 0.12% for the fusion instead of 1%, and at those points the two detect essentially the same malware (22.70% and 22.76%). The $+13.8$ points



(a) confusion matrices

(b) ROC (log FPR) and precision–recall curves

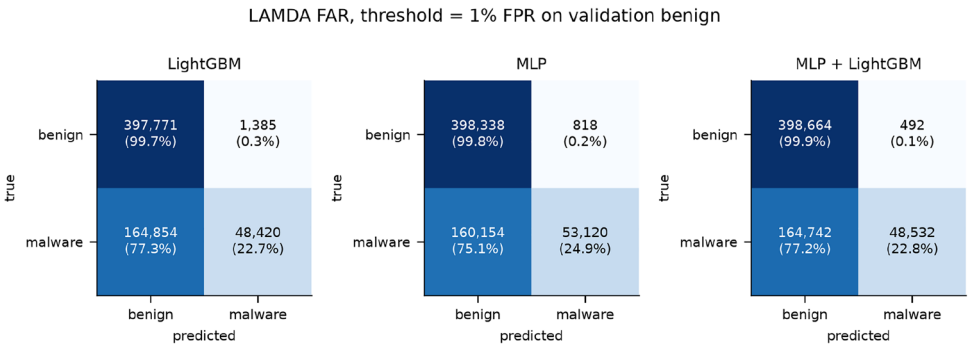
Fig. 12 LAMDA NEAR region: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves. Dotted lines mark the 1% and 0.1% FPR operating points.

are therefore a statement about *ranking quality* under a re-calibrated threshold, not a detection gain a deployment would obtain without re-calibration; the fusion ranks drifted malware better, but it is also the more conservative of the two, so at a fixed threshold it spends its advantage on a lower false-positive rate rather than on recall. The operational conclusion is that on a drifting stream the threshold, not the model, is the binding constraint, and any deployment of either model on LAMDA-like drift must re-calibrate periodically. This is the same requirement that Transcend[8] and TESSERACT[9] impose on evaluation, and that calibrated cost-sensitive triage applies in other domains[43].

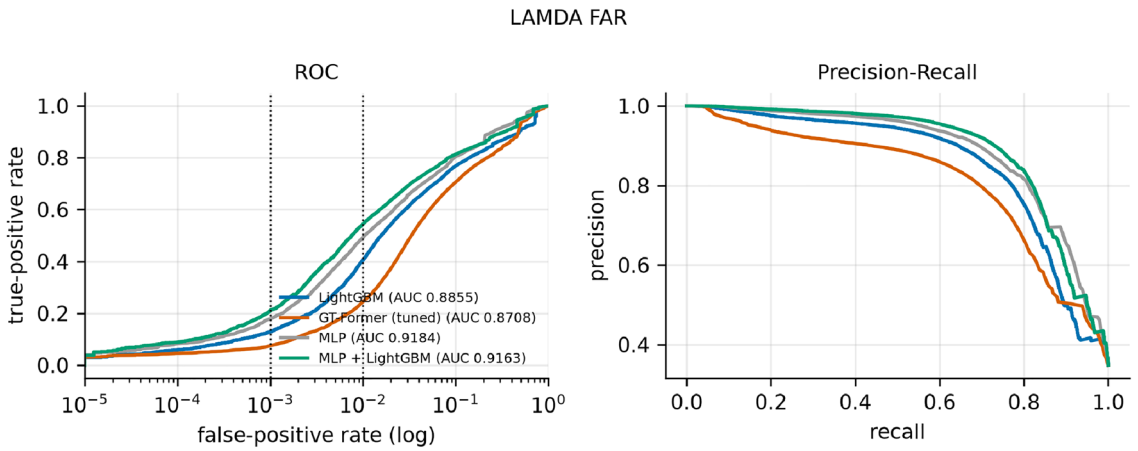
Year-by-year drift

With the threshold fixed on the IID benign files (Fig. 14), all models collapse in 2017 (TPR at 1% FPR below 0.12 for every model, F1 below 0.3), partially recover in 2019–2022, and fall to near zero in 2024–2025 when only 635 and 18 malware samples remain in the year. The MLP and ProtoCon-Net stay above LightGBM from 2019 onward at the IID-calibrated threshold (2020: 0.32–0.36 versus 0.27; 2021: 0.19–0.23 versus 0.12), which is the year-level view of the FAR result. The 2017 collapse coincides with the year in which the malware share of the corpus halves (21.5% versus 41–51% before) and the number of families quintuples (7,436 distinct AVClass2 labels in 2017 versus 1,454 in 2013, counting every distinct label including singletons; counting only families

https://doi.org/10.1038/s41598-026-75034-w



(a) confusion matrices



(b) ROC (log FPR) and precision–recall curves

Fig. 13 LAMDA FAR region: (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves. Dotted lines mark the 1% and 0.1% FPR operating points.

Table 7 LAMDA classical metrics at the validation-calibrated 1% FPR threshold. Fusion = rank fusion of the MLP and LightGBM.

Region	Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	FPR	F1	MCC	ROC-AUC	PR-AUC
IID	LightGBM	0.9660	0.9637	0.9870	0.9378	0.9896	0.0104	0.9618	0.9322	0.9963	0.9959
IID	MLP	0.9604	0.9575	0.9885	0.9239	0.9910	0.0090	0.9551	0.9214	0.9954	0.9950
IID	GT-Former (tuned)	0.9593	0.9563	0.9880	0.9221	0.9906	0.0094	0.9539	0.9193	0.9953	0.9947
IID	Fusion	0.9698	0.9679	0.9874	0.9460	0.9899	0.0101	0.9663	0.9397	0.9964	0.9961
NEAR	LightGBM	0.8196	0.7206	0.9738	0.4468	0.9944	0.0056	0.6126	0.5817	0.8296	0.8275
NEAR	MLP	0.8033	0.6978	0.9484	0.4059	0.9897	0.0103	0.5685	0.5369	0.8431	0.7981
NEAR	GT-Former (tuned)	0.7844	0.6697	0.9263	0.3525	0.9869	0.0131	0.5106	0.4842	0.8554	0.7955
NEAR	Fusion	0.8119	0.7088	0.9693	0.4240	0.9937	0.0063	0.5899	0.5618	0.8417	0.8242
FAR	LightGBM	0.7286	0.6118	0.9722	0.2270	0.9965	0.0035	0.3681	0.3897	0.8855	0.8571
FAR	MLP	0.7372	0.6235	0.9848	0.2491	0.9980	0.0020	0.3976	0.4153	0.9184	0.8883
FAR	GT-Former (tuned)	0.6992	0.5697	0.9551	0.1431	0.9964	0.0036	0.2488	0.2988	0.8708	0.8141
FAR	Fusion	0.7302	0.6132	0.9900	0.2276	0.9988	0.0012	0.3701	0.3973	0.9163	0.8939

Table 8 Inference cost on identical hardware and data. Throughput uses batch size 2,048; latency is a single file scored on its own, which is what a scanner processing files as they arrive actually pays.

Model	Device	Throughput (files/s)	Latency (μs/file)	Peak memory
LightGBM	CPU, 1 thread	16,780	147.4	–
LightGBM	CPU, 6 threads	74,340	177.8	–
LightGBM	CPU, 12 threads	**117,253**	237.6	–
GT-Former (tuned, 8.9 M)	GPU	18,017	3,222.7	536 MB
GT-Former (tuned, 8.9 M)	CPU, 12 threads	1,055	5,265.4	–
Fusion percentile lookup	CPU	–	0.288	–
Feature extraction (`thrember`)	CPU, 12 threads	1,625	≈615	–

Table 9 Add-one-in ablation on LAMDA, GT-Former, mean ± standard deviation over five seeds. TPR at the stated FPR, with the threshold taken from the benign files of the same region; novel-family and singleton strata on FAR at 1% FPR.

Variant	NEAR@1%	FAR@1%	NEAR@0.1%	FAR@0.1%	FAR novel	FAR singleton
plain (no objectives)	26.70 ± 2.46	27.76 ± 2.58	10.08 ± 2.26	8.64 ± 0.78	20.96 ± 2.96	16.38 ± 3.51
+ CAM only	27.34 ± 1.35	28.92 ± 2.69	12.33 ± 2.24	8.63 ± 0.79	21.19 ± 1.23	17.76 ± 4.24
+ consensus head only	26.90 ± 3.01	27.04 ± 4.69	10.46 ± 3.76	8.93 ± 1.08	19.86 ± 2.52	15.49 ± 6.69
+ family GRL only	32.22 ± 4.18	27.79 ± 6.70	6.88 ± 2.10	9.53 ± 2.11	19.32 ± 3.94	17.33 ± 9.39
full (all three)	30.83 ± 6.34	24.25 ± 4.09	4.43 ± 3.67	8.48 ± 1.49	17.26 ± 3.33	12.89 ± 4.70

Table 10 Win32 detection as a function of the fusion weight w on the tuned GT-Former, mean ± standard deviation over five seeds. $w = 0$ is LightGBM alone and $w = 1$ is the transformer alone.

w	TPR@1%	TPR@0.1%	Challenge@1%	Challenge@0.1%
0.0	96.85 ± 0.00	90.06 ± 0.00	63.41 ± 0.00	40.47 ± 0.00
0.1	97.17 ± 0.02	90.37 ± 0.05	66.13 ± 0.26	40.67 ± 0.14
0.2	97.27 ± 0.04	90.73 ± 0.08	68.09 ± 0.49	40.80 ± 0.23
0.3	97.27 ± 0.03	90.96 ± 0.07	68.73 ± 0.67	40.94 ± 0.33
0.4	97.22 ± 0.04	90.97 ± 0.09	68.68 ± 0.41	41.02 ± 0.41
0.5	97.15 ± 0.06	90.89 ± 0.06	68.20 ± 0.49	40.97 ± 0.82
0.6	97.04 ± 0.06	90.73 ± 0.08	67.24 ± 1.04	40.48 ± 1.75
0.7	96.87 ± 0.05	90.36 ± 0.09	64.50 ± 1.86	39.13 ± 3.36
0.8	96.66 ± 0.06	89.73 ± 0.18	60.71 ± 3.25	38.53 ± 3.93
0.9	96.39 ± 0.08	89.09 ± 0.22	56.22 ± 4.15	37.72 ± 3.71
1.0	96.07 ± 0.08	88.20 ± 0.35	52.31 ± 2.79	36.41 ± 3.72

Table 11 The same fusions under a threshold calibrated on the evaluation cohort (retrospective) and under a threshold fixed on validation data and applied unchanged (deployment). Realised FPR is what the deployment threshold actually produces on the test data at a 1% nominal budget.

		Retrospective	Deployment	Realised FPR
EMBER2024 Win32, challenge @1%	LightGBM	63.38	60.37	0.93%
	Fusion	68.96	61.30	0.87%
	difference	**+5.58**	**+0.93**	
LAMDA FAR, malware recall @1%	LightGBM	40.5	22.70	0.35%
	Fusion	54.4	22.76	0.12%
	difference	**+13.8**	**+0.06**	

with more than one member gives far smaller numbers, since 40.7% of all LAMDA malware are singletons), so it is a family-composition shift rather than a feature-space shift alone.

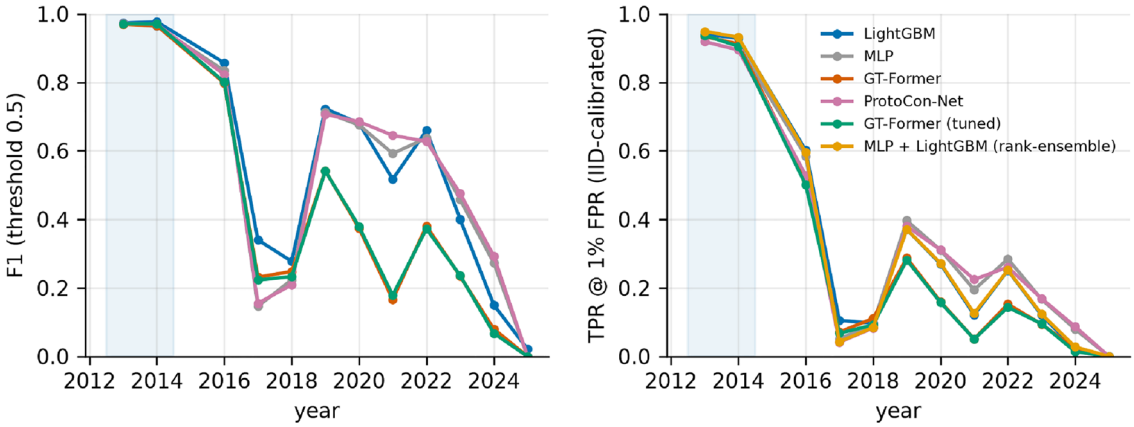


Fig. 14 LAMDA: F1 at threshold 0.5 (single models) and TPR at 1% FPR with the threshold calibrated on the IID benign files, per year; shaded years are the training years.

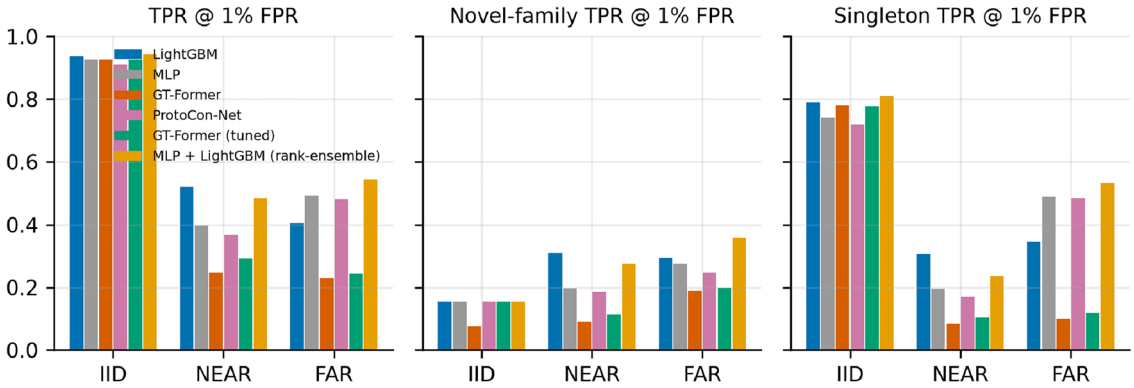


Fig. 15 LAMDA: TPR, novel-family TPR and singleton TPR at 1% FPR per temporal region.

Table 12 BODMAS test set (April–September 2020; 60,309 files, 27,701 malware). The tuned GT-Former is reported as mean ± standard deviation over five seeds; ensembles use seed 0. Nov. = novel-family TPR. Best value per column in bold.

Model	ROC-AUC	TPR@1%	TPR@0.1%	Nov.@1%	Nov.@0.1%
LightGBM (benchmark config.)	0.9999	0.9985	0.9892	0.9877	0.9465
MLP	0.9994	0.9864	0.9652	0.9621	0.9193
ProtoCon-Net	0.9993	0.9939	0.9823	0.9638	0.9374
GT-Former (default)	0.9993	0.9884	0.9770	0.9663	0.9440
GT-Former (tuned), five seeds	0.9996 ± 0.0003	0.9954 ± 0.0014	0.9872 ± 0.0008	0.9781 ± 0.0076	0.9572 (seed 0)
Fusion: tuned GT-Former + LightGBM	**0.9999**	**0.9985**	**0.9945**	0.9852	**0.9712**
Fusion: MLP + LightGBM	0.9999	0.9981	0.9894	**0.9918**	0.9514

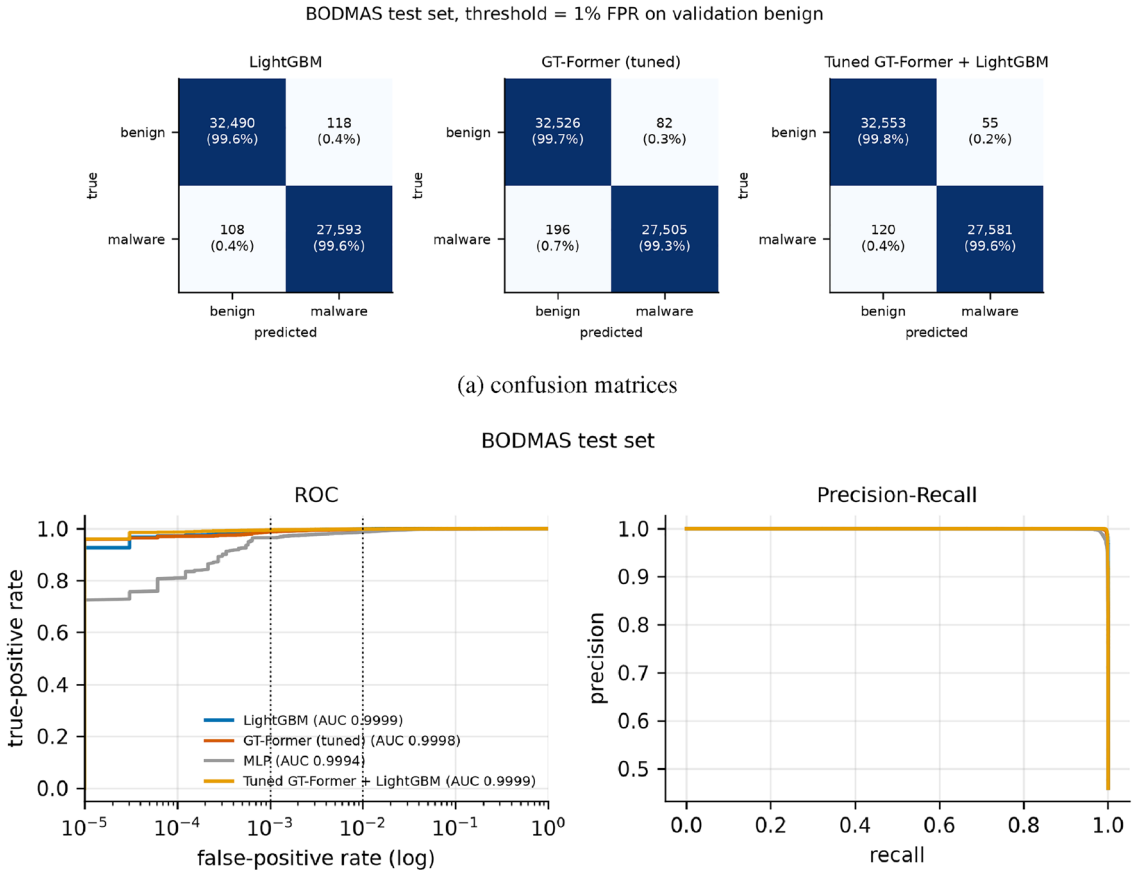
BODMAS

Detection on a third benchmark with version-2 features

BODMAS is easier than the other two: with only six months between the end of training and the last test month, every model exceeds 0.999 ROC-AUC (Table 12). It is nevertheless useful for two reasons. It uses EMBER *version-2* features (2,381 dimensions, LIEF-based), so it tests whether the group tokenisation of GT-Former transfers to a different feature layout, and it contains 1,215 test malware from families never seen in training.

Tuning, fusion and monthly drift

Optuna tuning is worth 0.7 points of TPR at 1% FPR and 1.0 point at 0.1% FPR for GT-Former (98.84% to 99.54 ± 0.14%; 97.70% to 98.72 ± 0.08%), and 1.2 points on novel families. The tuned transformer alone is still 0.3 points below LightGBM at 1% FPR (CI −0.23 to −0.12) and 1.2 points below on novel families, but the rank fusion of the two matches LightGBM exactly at 1% FPR (99.85%, CI −0.05 to +0.04) and is better at the strict 0.1% operating point (99.45% versus 98.92%) and on novel families at 0.1% FPR (97.12% versus 94.65%), which is the same complementarity as on EMBER2024. Month by month (Fig. 17), LightGBM, the tuned GT-Former and the fusion stay above 99% TPR at the March-calibrated threshold until September 2020, where the tree model dips to 98.4% and the fusion to 98.9%; the MLP drifts to 98.0% from August; and the single-seed default GT-Former falls below 90% in July, beneath the axis of the figure, and recovers only to 94–95% in August and September, the same single-run instability that Section 4.1 documents on the small EMBER formats and that the five-seed tuned model does not show. Novel-family detection moves more for every model (94–99% by month for the tuned models, and below 80% for the default GT-Former from June). Figure 16 shows the confusion matrices and curves at the March-calibrated threshold and Table 13 the classical metrics. BODMAS therefore confirms that the tokenisation of GT-Former transfers to the version-2 layout and that the deep/tree fusion is the safest default, while adding little discriminative pressure because the benchmark is close to saturation.



(a) confusion matrices

(b) ROC (log FPR) and precision–recall curves

Fig. 16 BODMAS test set (April–September 2020): (**a**) confusion matrices of LightGBM, the tuned GT-Former and the fusion at the validation-calibrated 1% FPR threshold, (**b**) ROC and precision–recall curves. Dotted lines mark the 1% and 0.1% FPR operating points.

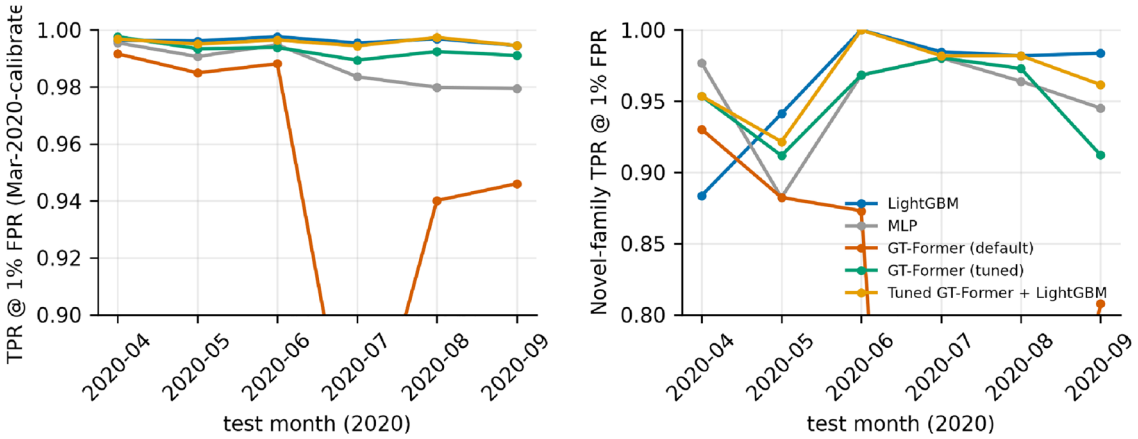


Fig. 17 BODMAS: TPR and novel-family TPR at 1% FPR per test month (threshold calibrated on March 2020 benign files).

Table 13 BODMAS classical metrics at the validation-calibrated 1% FPR threshold (test set April–September 2020). Fusion = rank fusion of the tuned GT-Former and LightGBM.

Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	FPR	F1	MCC	ROC-AUC	PR-AUC
LightGBM	0.9963	0.9962	0.9957	0.9961	0.9964	0.0036	0.9959	0.9925	0.9999	0.9999
MLP	0.9886	0.9885	0.9876	0.9875	0.9895	0.0105	0.9875	0.9770	0.9994	0.9993
GT-Former (tuned)	0.9954	0.9952	0.9970	0.9929	0.9975	0.0025	0.9950	0.9907	0.9998	0.9997
Fusion	0.9971	0.9970	0.9980	0.9957	0.9983	0.0017	0.9968	0.9942	0.9999	0.9999

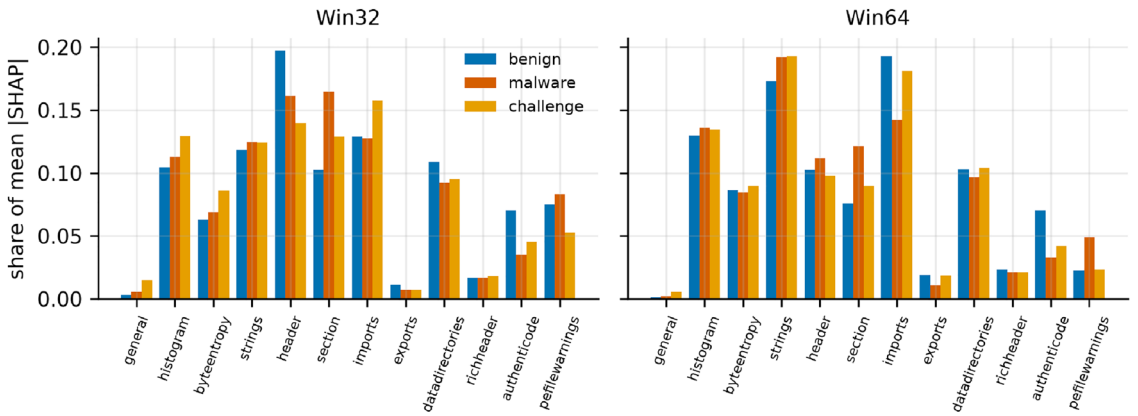


Fig. 18 LightGBM SHAP attribution share per feature group on Win32 and Win64 for benign files, ordinary malware and challenge files.

Explainability: what separates evasive from ordinary malware

We explain the two strongest EMBER2024 models with complementary tools: SHAP TreeExplainer values on the LightGBM benchmark (3,000 test benign and 3,000 test malware files per format plus every challenge file of that format; Figures 18 and 19 and the last-layer CLS attention of GT-Former over the twelve group tokens (Fig. 20).

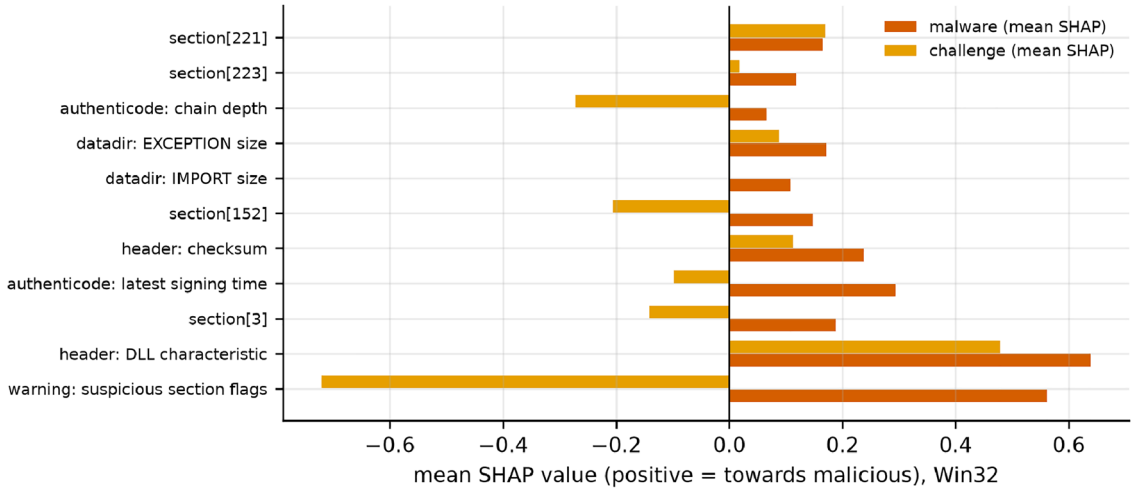


Fig. 19 Sign flip of the most important LightGBM features between ordinary and challenge malware (Win32, mean SHAP value).

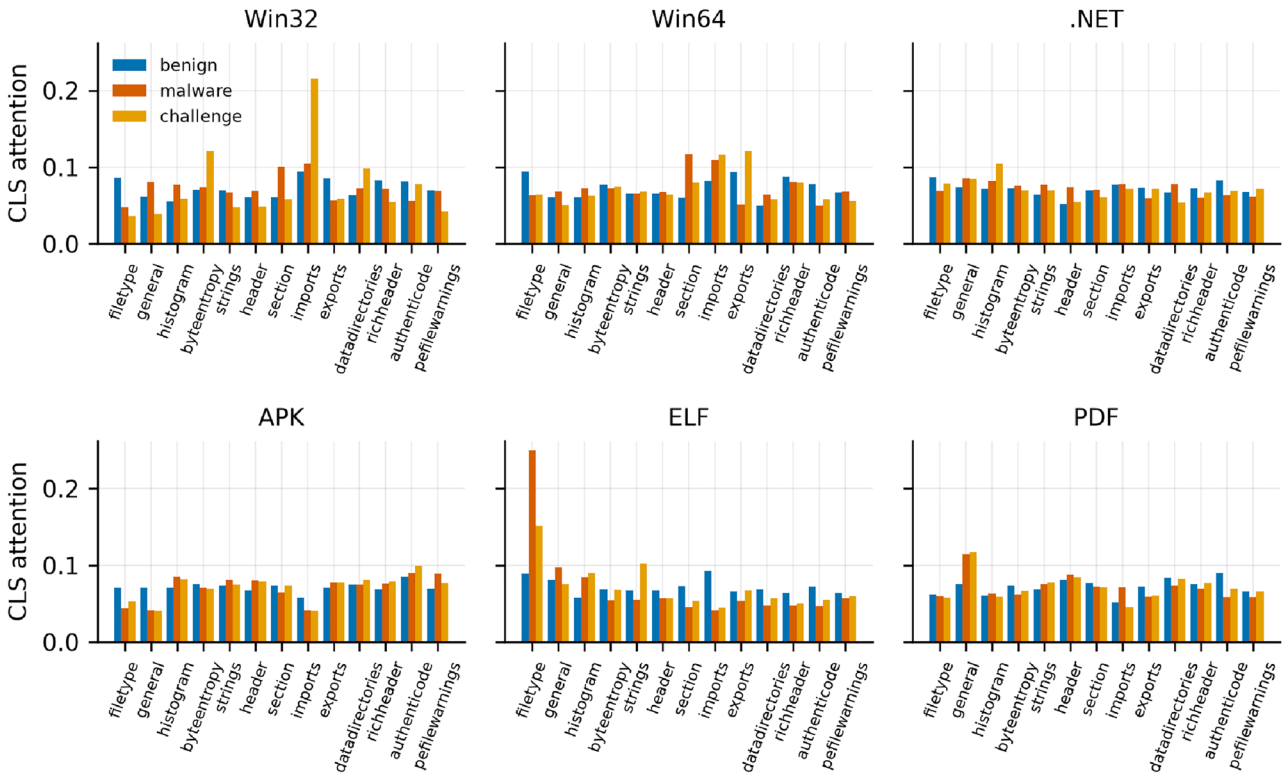


Fig. 20 GT-Former last-layer CLS attention over the twelve group tokens for benign files, ordinary malware and challenge files.

Group-level attribution (SHAP)

On Win32 the benchmark spreads its attribution over the header (16–20%), sections (10–17%), imports (13–16%), strings (12%) and byte-histogram (11–13%) groups. Two shifts distinguish challenge files from ordinary malware: attribution to the *imports* group rises from 12.8% to 15.8% and to the byte-entropy group from 6.9% to

8.6%, while the *pefile warnings* group falls from 8.3% to 5.3% and *sections* from 16.5% to 12.9%. On Win64 the same pattern appears (imports 14.2% to 18.1%, sections 12.1% to 8.9%). For the non-PE formats the model can only use the format-agnostic groups (histogram, byte-entropy, strings) and the attribution profile of challenge files is indistinguishable from ordinary malware, which is consistent with their low challenge-set detection.

Feature-level attribution and the direction of the evidence

The single most important LightGBM feature on Win32 and Win64 is the pefile warning "Suspicious flags set for section" (mean |SHAP| 1.26 on Win32 malware). For ordinary malware it pushes *towards* malicious (mean SHAP +0.56); for challenge files its mean contribution is *negative* (−0.72): evasive files are, on average, well-formed PEs that do not trigger the warning, and the model reads the absence of the warning as evidence of benignness. The same sign flip occurs for the Authenticode features (latest signing time and certificate-chain depth: +0.29 on ordinary malware, −0.10 and −0.27 on challenge files), and for the `DLL` image characteristic the positive contribution shrinks from +0.64 to +0.48 (Fig. 19). These are associations in the model's attributions, not statements about the files themselves: a negative Authenticode attribution means the model treats those feature values as evidence of benignness on challenge files, and does not by itself establish that the files are signed. The pattern is consistent with the reading that the benchmark's strongest evidence is *structural sloppiness* (suspicious section flags, missing signatures, unusual checksums and linker versions) and that evasive malware avoids it; the causal version of that claim is supported not by the attributions but by the group knock-out experiment, in which removing the packaging groups at training time changes challenge detection while leaving ordinary detection almost unchanged.

Attention of GT-Former

For Win32 benign files the CLS token attends almost uniformly to the twelve groups (0.056–0.094 per token). For ordinary malware it concentrates on imports (0.101) and exports (0.105). For challenge files attention on the exports token doubles (0.215) and attention on strings (0.121) and the rich header (0.098) rises, while attention on the byte histogram (0.039) and general features (0.036) falls (Fig. 20). GT-Former therefore looks for evasive malware in *what the binary exposes* (export table, embedded strings, toolchain fingerprint) rather than in byte statistics, which is a plausible reason why its errors are complementary to the tree model's and why the rank ensemble recovers evasive files at a fixed false-positive budget. On ELF the general token (file size, entropy, is_pe) receives 25% of the malware attention, and on PDF the byte histogram dominates for both ordinary and challenge malware.

Mixture-of-experts routing

The FR-MoE gate learns format-specific mixtures (Win64: 37% expert 4 and 25% expert 3; APK: 24% expert 5 and 22% expert 2; .NET: 36% expert 4), confirming that the file-type signal is used, but no expert specialises exclusively on one format, which matches the finding that FR-MoE gives no accuracy advantage over the flat MLP.

Discussion

Why gradient boosting still wins alone in distribution

The deficit of the deep models against LightGBM is small in ROC-AUC (0.9975 against 0.9982 on Win32) and concentrated at strict FPR, and it has two separable causes. The first is variance. At 0.1% FPR the threshold is

fixed by a few hundred benign files on Win32 and by a handful on ELF and PDF, and the deep models' benign score tails move between seeds in a way the deterministic tree's do not: the five-seed spread of the tuned GT-Former on Win32 challenge detection at 1% FPR is ±3.1 points against a mean deficit to LightGBM of about ten points (Section 4.1), and on ELF the spread (±0.14) is as large as the deficit itself. The second is representation. EMBER version-3 and Drebin features are heavy-tailed counts, hashed bins and binary indicators; a tree places an exact threshold on a raw value (a section flag set, a specific import present, an entropy above a cut), which is precisely the kind of evidence that separates the last 0.1% of benign files from malware, whereas a deep model sees a standardised, fp16-quantised and non-linearly squashed version of the same value and must approximate the cut with a smooth function. The deep models also receive a fixed budget of 8–12 epochs on one GPU. The evidence that the two causes are distinct is that the fusion, which cannot reduce the transformer's variance, still recovers the strict-FPR gap (Section Sensitivity of the fusion to its weighting: the fused TPR at 0.1% FPR exceeds the tree's for every weight up to 0.7), because the tree supplies the exact thresholds and the transformer supplies evidence the tree does not use; and that the tuning study, which changes representation capacity but not the training budget, narrows the gap by less than a point.

Why fusion helps where it matters

The deep model's errors are partly independent of the tree model's, since Section Explainability: what separates evasive from ordinary malware shows that the two use different evidence; averaging ranks recovers evasive and drifted files that the tree ranks below the operating threshold. The gain is largest exactly where single models are weakest: the challenge set on EMBER2024 and the far-future, singleton-heavy years of LAMDA (+13.8 and +18.7 points at region-calibrated thresholds; Section Retrospective and deployment operating points gives the deployment figures). Three pieces of evidence connect the gain to the design rather than to averaging in general. The weight sweep (Section Sensitivity of the fusion to its weighting) shows that the optimum is flat between $w = 0.2$ and 0.5 and that pushing weight towards the transformer destroys it, so the fusion works because a stable, exact-threshold model anchors a higher-variance model with different evidence, not because two models are better than one; the three-deep-model ensemble, whose members share evidence, is worse than LightGBM alone. The attention and SHAP analyses (Section Explainability: what separates evasive from ordinary malware) show that the two partners weigh different feature groups on challenge files. And the ablations show that none of the gain comes from the auxiliary objectives, which are absent from the fused model. The sources of variation in the fusion result are the transformer seed (negligible at $w = 0.5$ on Win32, ±0.5 points, but ±0.14 in TPR at 0.1% FPR on ELF), the threshold (Section Retrospective and deployment operating points) and, on LAMDA, the region composition; we report all three rather than the most favourable.

Why the deep model's robustness to drift differs between benchmarks

On LAMDA the MLP degrades more gracefully than LightGBM over 2018–2025, while on the twelve EMBER2024 test weeks the tree model is at least as stable. The FAR region of LAMDA is dominated by singletons (131,648 of 213,274 malware) and by a benign-heavy class balance; a smooth, high-capacity decision function generalises to these better than sharp thresholds tuned on 2013–2014. The twelve EMBER2024 test weeks contain too little drift to show the same effect. The three benchmarks therefore test different kinds of shift and the framework handles them differently. EMBER2024 tests *evasion* at almost no temporal distance (85% of challenge files are contemporaneous with training), and the fusion's gain there comes from complementary evidence at a fixed budget. LAMDA tests *temporal and family-composition* shift over twelve years, where the fusion's ranking advantage is real but a fixed threshold does not survive (Section Retrospective and deployment operating points) and periodic re-calibration is the binding requirement. BODMAS tests *feature-layout transfer*: the same tokenisation applied to version-2 features gives the same complementarity, but at six months' distance the benchmark is close to saturation and discriminates little. Nothing in this design transfers a model across

benchmarks: the EMBER2024 models are not applied to LAMDA or BODMAS, because the feature spaces differ, and cross-dataset generalisation in that sense is not evaluated here. Three limitations of the novel-family and drift evaluation follow. Within-format novelty on EMBER2024 is weaker than global novelty for jointly trained models (Section EMBER2024); the EMBER2024 conclusion that the tree is at least as drift-stable as the deep models does not transfer to LAMDA's binary Drebin features, where the smooth model degrades more gracefully, so drift robustness is a property of the model–feature pair rather than of the architecture; and the 2024–2025 years of LAMDA contain too few malware samples for the year-level numbers to be reliable.

Why the consensus and family objectives did not pay for themselves

Challenge files are low-consensus *at their final rescan*, but the training-time consensus of ordinary malware is only weakly related to how a *different* file evades, so consensus supervision reshapes the boundary for the wrong files. With five seeds per variant (Section Activating each objective on its own) no single objective is distinguishable from the plain model on LAMDA, and the only effect that exceeds seed noise is that the three together lower TPR at the strictest budget; the single-seed EMBER2024 ablation is consistent with this but cannot establish per-objective effects. We therefore make no claim about any objective on its own, and in particular withdraw the attribution of the loss to the family-adversarial head that the submitted version made. A more promising direction is to use consensus for *sample selection* (curriculum or hard-example mining) and family labels for *evaluation*, not for invariance.

Benchmark hygiene

Every paper that trained on the raw EMBER2024 Hugging Face files trained on each record twice. This creates no train/test leakage, but it doubles reported dataset sizes and epoch costs and changes early-stopping behaviour. We release the SHA-256 keep-masks and the exact LAMDA split indices.

Limitations

The study uses static features only and three benchmarks, one of which (BODMAS) is near saturation. Antivirus consensus is a noisy proxy for evasiveness, and the ClarAVy and AVClass2 confidences were not exploited. We did not evaluate functionality-preserving adversarial attacks. The deep models were trained for a fixed short budget on one GPU, and although Optuna tuning narrowed the gap to LightGBM it did not close it. The tuning budget was asymmetric: only GT-Former received a hyper-parameter search, with ROC-AUC rather than a strict-FPR metric as its objective, while LightGBM, the MLP and FR-MoE ran at their published or default settings; the comparison among the deep encoders is therefore not budget-matched, and the parameter-matched MLP control supports the tokenisation claim only for the default GT-Former. Recent tabular deep learners (TabM, ModernNCA) were not trained on these corpora, and C-GUARD is compared on its published counts only. Novel-family detection on EMBER2024 is defined within format, which is a weaker criterion than global novelty for jointly trained models. The smallest EMBER formats (ELF and PDF) and GT-Former on LAMDA give seed-unstable results, and the 2024–2025 years of LAMDA contain too few malware samples for reliable TPR estimates.

Conclusion

Under an operational lens the picture on EMBER2024, LAMDA and BODMAS is consistent. The gradient-boosted benchmark is still the best single model in distribution; the deep architectures we propose come close (Optuna tuning brings GT-Former within 1.5 points of LightGBM at 0.1% FPR on Win32 and within 0.3 points at 1% FPR on BODMAS) but do not overtake it; the consensus- and family-aware objectives we designed do not

help, and in the configurations we tested no objective on its own is distinguishable from the plain model while the three together lower detection at the strictest budget; and a rank fusion of the benchmark with one deep model of a different inductive bias is the best detector we found for evasive malware, novel families and far-future drift. That last claim must be read with its threshold: at thresholds calibrated on the evaluation region the gains are statistically significant on all three benchmarks (Win32 challenge detection +5.6 points, LAMDA far-future detection +13.8), and at thresholds fixed on validation data and applied unchanged the EMBER2024 gain survives at +0.9 points while on LAMDA twelve years of drift move both models far below their budget and the gain vanishes, so on a drifting stream re-calibration is the binding requirement. On BODMAS the fusion matches the tree at 1% FPR and improves it at 0.1%, and on LAMDA the MLP alone retains a marginally higher far-future ROC-AUC than the fusion. The study is limited to static features, to three benchmarks of which one is near saturation, to an asymmetric tuning budget, and to seed-unstable results on the smallest formats; it does not evaluate functionality-preserving adversarial attacks. Explainability shows that the benchmark's evidence for maliciousness is structural sloppiness that evasive malware avoids, and that the transformer looks elsewhere (export tables, strings, toolchain fingerprints), which is why the two are complementary. We release the protocol, the hygiene fixes and every score file so that future work can be compared at the same operating points.

Author contributions Mohammed Tawfik: Conceptualization, Methodology, Software, Formal Analysis, Investigation, Writing—Original Draft. Belal Al-sellami: Methodology, Data Curation, Validation, Writing—Review & Editing, Corresponding Author. Mohamed Heshmat: Investigation, Formal Analysis, Visualization, Writing—Review & Editing. Islam S. Fathi: Validation, Resources, Writing—Review & Editing. Warda M. Shaban: Supervision, Project Administration, Writing—Review & Editing. All authors reviewed and approved the final manuscript

Funding This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability The three benchmarks are publicly available from their authors. • EMBER2024 (Apache-2.0): https://huggingface.co/datasets/joyce8/EMBER2024 • LAMDA (MIT): https://huggingface.co/datasets/IQSeC-Lab/LAMDA • BODMAS: https://whyisyoung.github.io/BODMAS/ The code of this paper (feature vectorisation, de-duplication masks, the exact LAMDA and BODMAS split indices, the LightGBM variants, the GT-Former, FR-MoE and ProtoCon-Net implementations, Optuna tuning, rank fusion, bootstrap tests, tables, figures and the SHAP and attention analyses), all trained models and all score files for random seeds 0–4 are released at https://github.com/kmkholm/GTFormer (score files and trained models are attached to the repository release).

Declarations

Competing interests The authors declare no competing interests.

Ethical approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Joyce, R. J. et al. EMBER2024 – a benchmark dataset for holistic evaluation of malware classifiers. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, https://doi.org/10.1145/3711896.3737431 (2025).
2. Haque, M. A. LAMDA: A longitudinal Android malware benchmark for concept drift analysis. In Proceedings of the 14th International Conference on Learning Representations (ICLR (2026).
3. Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A. & Wang, G. BODMAS: An open dataset for learning based temporal analysis of PE malware. In *Proceedings of the IEEE Security and Privacy Workshops (Deep Learning and Security, DLS)*, https://doi.org/10.1109/SPW53761.2021.00020 (2021).
4. Aljarrah, N., Shehadeh, H. H., Obeidat, R. A. & Tawfik, M. Cross-attention feature fusion for interpretable zero-day malware detection. *Statistics, Optimization & Information Computing***15**, 2164–2178, https://doi.org/10.19139/soic-2310-5070-2900 (2026).
5. Tawfik, M. et al. Few-shot Android malware classification with quantum-enhanced prototypical learning and drift detection. *Sci. Rep.* **16**, 10744. https://doi.org/10.1038/s41598-026-45738-0 (2026).
6. Anderson, H. S. & Roth, P. EMBER: An open dataset for training static PE malware machine learning models. Tech. Rep., Endgame (2018). Technical report.
7. Ke, G. LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems Vol. 30 (2017).
8. Jordaney, R. Transcend: Detecting concept drift in malware classification models. In *Proceedings of the 26th USENIX Security Symposium* 625–642 (2017).
9. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J. & Cavallaro, L. TESSERACT: Eliminating experimental bias in malware classification across space and time. In *Proceedings of the 28th USENIX Security Symposium*, 729–746 (2019).
10. Chow, T. et al. Beyond the TESSERACT: Trustworthy dataset curation for sound evaluations of Android malware classifiers. *In Proceedings of the IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)* https://doi.org/10.1109/satml68715.2026.00052 (2026).
11. Horvath, D.-C. & Zsigmond, I. A comparative benchmark of machine learning models for static malware analysis: From EMBER 2018 to the challenges of 2024. In *Proceedings of the 18th International Conference on Agents and Artificial Intelligence (ICAART)*, vol. 3, 2284–2291, https://doi.org/10.5220/0014218700004052 (2026).
12. Mann, S., Abbas, S. & Sultana, A. Temporal generalization of static malware detectors: A large-scale empirical study. In *Proceedings of the 2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies (ICAISET)*, https://doi.org/10.1109/ICAISET66439.2026.11542134 (2026).
13. Kalný, T., Jureček, M. & Stamp, M. Detecting concept drift in evolving malware families using rule-based classifier representations. In *Proceedings of the 23rd International Conference on Security and Cryptography (SECRYPT)*, 922–931, https://doi.org/10.5220/0015184000004103 (2026).
14. Park, J. et al. TabCL: Continual malware classification with tabular-aware generation. In *Advances in Knowledge Discovery and Data Mining (PAKDD 2026), Lecture Notes in Computer Science*, https://doi.org/10.1007/978-981-92-1465-5_43 (Springer, 2026).
15. Raza, M. M. et al. Evolutionary feature engineering meets AutoML: A feature-efficient framework for malware detection. *IEEE Access* https://doi.org/10.1109/ACCESS.2026.3699588 (2026).
16. Mossadaq, A. & Mandar, M. A deep learning-based malware triage framework with uncertainty-aware detection for SOC environments. In *Proceedings of the 2026 International Conference on Cybersecurity, Digital Forensics, and AI Applications (ICCSDFAI)*, https://doi.org/10.1109/ICCSDFAI70505.2026.11648045 (2026).
17. Imran, M., Al-Essa, M., Franchini, F. & Pirlo, G. C-GUARD: Context-adaptive conformal gating for improving robustness against evasive Windows PE malware. *Future Internet* **18**, 431. https://doi.org/10.3390/fi18080431 (2026).
18. Nguyen Kim, T., Doan Trung, S. & Pham, N. PE-MILCon: Multiple-instance learning with contrastive multi-view representation for static Windows PE malware detection. *Computers, Materials & Continua* https://doi.org/10.32604/cmc.2026.084268 (2026).
19. Rahman, M. S., Coull, S., Yu, Q. & Wright, M. MADAR: Efficient continual learning for malware analysis with distribution-aware replay. In *Proceedings of Machine Learning Research*, vol. 299 (2025).
20. Qiu, L. et al. PLCDroid: Enhancing Android malware detection by mitigating pseudo-label noise in the presence of concept drift. *Comput. J.* https://doi.org/10.1093/comjnl/bxag021 (2026).
21. Ghiani, D. et al. Regression-aware continual learning for Android malware detection. *IEEE Trans. Inf. Forensics Secur.* https://doi.org/10.1109/TIFS.2026.3714132 (2026).
22. Minnei, L. et al. Label-efficient training updates for malware detection over time. In *CEUR Workshop Proceedings* (2025). Volume and pages to be completed.
23. Huynh, T., Huynh, D., Nguyen, M. N. T. & Trinh, V. Temporal degradation in machine learning-based malware detection: A multi-dataset, multi-year empirical study. *IEEE Access* https://doi.org/10.1109/ACCESS.2026.3706553 (2026).

24. Wang, Q. et al. Measurement of malware family classification on a large-scale real-world dataset. In *Proceedings of the IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, https://doi.org/10.1109/TrustCom56396.2022.00196 (2022).
25. Lu, Q., Zhang, H., Kinawi, H. & Niu, D. Self-attentive models for real-time malware classification. *IEEE Access* **10**, 95970–95985. https://doi.org/10.1109/ACCESS.2022.3202952 (2022).
26. Yan, J., Jia, X., Ying, L. & Su, P. Understanding and mitigating label bias in malware classification: An empirical study. In *Proceedings of the IEEE International Conference on Software Quality, Reliability and Security (QRS)*, https://doi.org/10.1109/QRS57517.2022.00057 (2022).
27. Yumlembam, R., Issac, B. & Jacob, S. M. Enhancing decision-making in Windows PE malware classification during dataset shifts with uncertainty estimation. *Knowl.-Based Syst.* https://doi.org/10.1016/j.knosys.2025.114723 (2025).
28. Truong, D. Cross-dataset feature discriminability in static malware detection: A CVFR-based study across four heterogeneous benchmarks. *IEEE Access* https://doi.org/10.1109/ACCESS.2026.3717791 (2026).
29. Tawfik, M., Abu-Ein, A. A., Noaman, H. M., Abdelhaliem, A. H. & Fathi, I. S. FedMedSecure: federated few-shot learning with cross-attention mechanisms and explainable AI for collaborative healthcare cybersecurity. *Sci. Rep.* **15**, 40050. https://doi.org/10.1038/s41598-025-25107-z (2025).
30. Tawfik, M., Abu-Ein, A. A., Abdelhaliem, A. H., Al-Sharo, Y. M. & Fathi, I. S. Explainable few-shot learning with modern BERT for detecting emerging phishing attacks using XF PhishBERT. *Sci. Rep.* **15**, 42821. https://doi.org/10.1038/s41598-025-27500-0 (2025).
31. Khosla, P. Supervised contrastive learning. In Advances in Neural Information Processing Systems Vol. 33 (2020).
32. Ganin, Y. et al. Domain-adversarial training of neural networks. *Journal of Machine Learning Research***x17**, 1–35 (2016).
33. Singh, P., Borgohain, S. K. & Kumar, J. Optimized feature representation and selection for malware detection using portable executable headers and machine learning. *J. Computer Virology Hacking Tech.* https://doi.org/10.1007/s11416-025-00555-3 (2025).
34. Singh, P., Borgohain, S. K., Sarkar, A. K., Kumar, J. & Sharma, L. D. Feed-forward deep neural network (ffdnn)-based deep features for static malware detection. *Int. J. Intell. Syst.* https://doi.org/10.1155/2023/9544481 (2023).
35. Singh, P., Borgohain, S. K., Sharma, L. D. & Kumar, J. Minimized feature overhead malware detection machine learning model employing mrmr-based ranking. *Concurrency and Computation: Practice and Experience* https://doi.org/10.1002/cpe.6992 (2022).
36. Singh, P., Borgohain, S. K. & Kumar, J. Performance enhancement of SVM-based ml malware detection model using data preprocessing. In *2022 2nd International Conference on Emerging Frontiers in Electrical and Electronic Technologies (ICEFEET)*, https://doi.org/10.1109/ICEFEET51821.2022.9848192 (2022).
37. Singh, P., Borgohain, S. K. & Kumar, J. Investigation and pre-processing of CLaMP mlaware dataset for machine learning models. In *2022 6th International Conference on Electronics, Communication and Aerospace Technology*, https://doi.org/10.1109/ICECA55336.2022.10009153 (2022).
38. Beg, R., Pateriya, R. K. & Tomar, D. S. ACMFNN: A novel design of an augmented convolutional model for intelligent cross-domain malware localization via forensic neural networks. *IEEE Access* https://doi.org/10.1109/ACCESS.2023.3305274 (2023).
39. Beg, R., Pateriya, R. K. & Tomar, D. S. Design of an iterative method for malware detection using autoencoders and hybrid machine learning models. *IEEE Access* https://doi.org/10.1109/ACCESS.2024.3491185 (2024).
40. Beg, R., Pateriya, R. K., Tomar, D. S. & Solanki, S. Detecting malware evidences through static and dynamic information using extreme machine learning for forensic analysis. *Discover Computing* https://doi.org/10.1007/s10791-025-09842-5 (2025).
41. Beg, R. et al. Design of an integrated evidence-driven few-shot meta-learning for zero-day malware detection and forensic attributions. *Sci. Rep.* https://doi.org/10.1038/s41598-026-43745-9 (2026).
42. Shahiduzzaman, M. et al. Attention-guided quantum-inspired extreme learning machine with grey wolf optimization for rare-attack detection in IIoT networks. *Discover Artificial Intelligence* https://doi.org/10.1007/s44163-026-01704-3 (2026).
43. Hossain, M. S. et al. TEMPLAR fraud a verifier grounded calibrated and cost sensitive framework for transaction fraud detection. *Discover Data* https://doi.org/10.1007/s44248-026-00117-y (2026).
44. Shehan, M. S. H. et al. Attention guided hybrid fusion of MobileNetV3 DenseNet121 and DINOv2 for five class white blood cell classification. *Discover Analytics* https://doi.org/10.1007/s44257-026-00086-1 (2026).
45. Rahman, M. S. et al. Self supervised iBOT vision transformer framework for data efficient white blood cell classification. *Discover Artificial Intelligence* https://doi.org/10.1007/s44163-026-01185-4 (2026).
46. Vaswani, A. Attention is all you need. In Advances in Neural Information Processing Systems Vol. 30 (2017).
47. Akiba, T., Sano, S., Yanase, T., Ohta, T. & Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2623–2631, https://doi.org/10.1145/3292500.3330701 (2019).
48. Gorishniy, Y., Kotelnikov, A. & Babenko, A. TabM: Advancing tabular deep learning with parameter-efficient ensembling. In *International Conference on Learning Representations (ICLR)* (2025).

49. Ye, H.-J., Yin, H.-H., Zhan, D.-C. & Chao, W.-L. Revisiting nearest neighbor for tabular data: A deep tabular baseline two decades later. In *International Conference on Learning Representations (ICLR)* (2025).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Mohammed Tawfik[1] · Belal Al-sellami[2] · Mohamed Heshmat[3] · Islam S. Fathi[4] · Warda M. Shaban[5]

✉ Belal Al-sellami
alsellamibelal@gmail.com

[1] Department of Cyber Security, Faculty of Information Technology, Ajloun National University, P.O.43, 26810 Ajloun, Jordan

[2] Faculty of Computer and Information Technology, Sana'a University, Sana'a, Yemen

[3] Department of Data Science & Artificial Intelligence, Faculty of Information Technology, Ajloun National University, Ajloun, Jordan

[4] Department of Computer Science, Faculty of Information Technology, Ajloun National University, Ajloun, Jordan

[5] Faculty of Artificial Intelligence and Informatics, Horus University, Damietta, Egypt